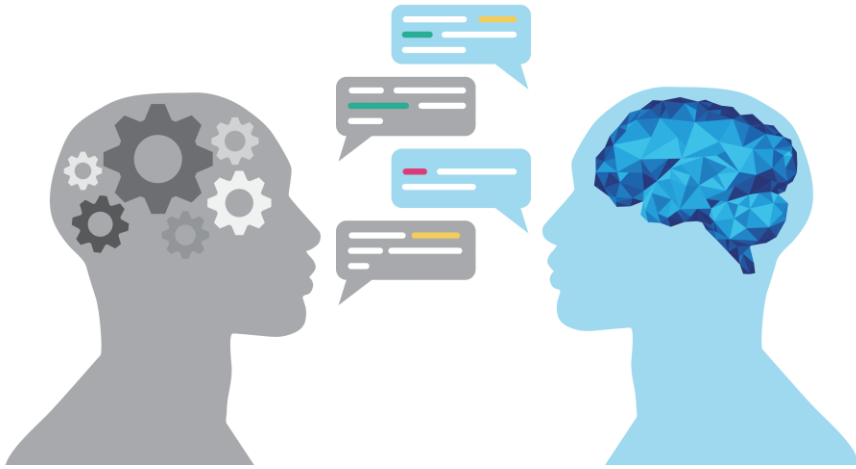




Road to ChatGPT: Basics of AI in Language Processing Day 3

A/Prof Edmund Low
National University of Singapore
edmundlow@nus.edu.sg

Recap – Day 2

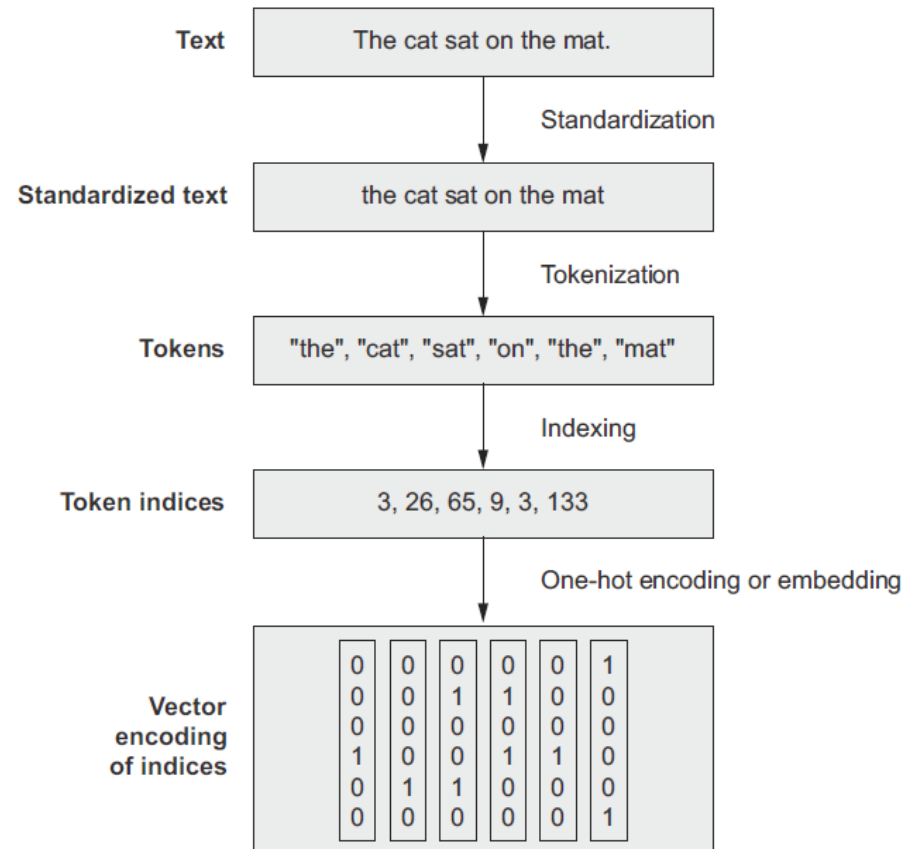
- Introduction to Natural Language Processing
- Pre-process text data for NLP
- Predicting sentiment from words
- Embeddings and improving model fit

Agenda

- Another way of modelling words – sequence models
- Introducing Transformers
- Using transformers to classify text

Another approach to modelling language

- In our previous classes, we've looked at how to tokenize text and sequence them into strings of integers
- Subsequently, we considered embeddings, which allows us to cluster words with similar meanings together, so that we can more accurately predict sentiment



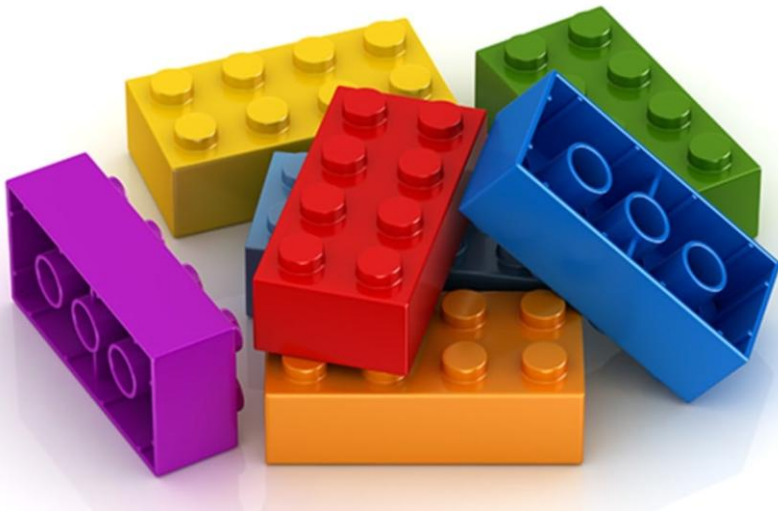
Another approach to modelling language

- We found that, especially for the embeddings model, that it worked really well for predicting sentiment
- But ultimately, there is a limitation to modelling text as a bag of words, since the sequence of words can also be important in determining their overall meaning
- For example, the individual words “career” and “performance” are essentially neutral from a sentiment perspective, but when we put them together, there is clear connotation of something that’s positive

Sequence models

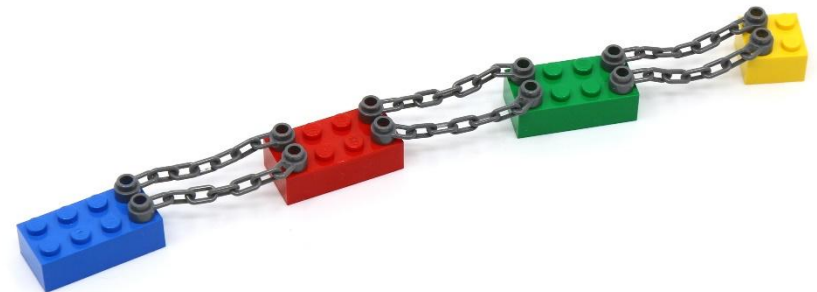
- To take sequences like this into consideration, another approach to processing text is required, aka sequence models
- How to implement a sequence model is not wholly separate from what we have seen so far; we would still need to first represent our input samples (e.g. reviews) as sequences of integers or embeddings
- But now, we will feed these integers/embeddings into a stack of layers to find correlations between adjacent vectors (i.e. word order relationships)

Sequence models



Bag of words: looking at each “block” individually, with no concern about the order of the blocks

Sequence: looking at all the blocks together, and the order in which they appear



The Transformer



- One such “stack of layers” that is able to do this cross-correlation between adjacent vectors is the Transformer
- First proposed in 2017, in the paper “Attention is all you need” by Vaswani et al
- This (relatively) new model architecture now predominates in most natural language processing tasks

The Transformer

- It is the model that underpins larger language models like ChatGPT. Below is the timeline tracking the evolution from Transformers, to ChatGPT (in ChatGPT's own words!)

Year	What happened
2017	Transformers were first introduced in a 2017 paper by Vaswani et al. They were designed to address the limitations of previous language models and were used for a variety of natural language processing tasks
2018	Google introduced Bidirectional Encoder Representations from Transformers (BERT), a pre-trained language model that uses transformer-based architecture. BERT was trained on a large corpus of text and fine-tuned for specific downstream tasks, such as question answering and text classification.

The Transformer

Year	What happened
2019	OpenAI introduced the second version of their Generative Pre-trained Transformer (GPT-2) language model. GPT-2 was also pre-trained on a large corpus of text and was capable of generating high-quality natural language text in response to prompts.
2020	OpenAI introduced the third version of their GPT series, GPT-3 , which is currently the largest language model in existence with over 175 billion parameters. GPT-3 is capable of generating highly coherent and contextually appropriate text and has been used for various natural language processing tasks, including language translation, chatbots, and content creation

The Transformer

Year	What happened
2022	OpenAI released a new version of their GPT language model, called ChatGPT, which is specifically designed for conversational applications such as chatbots. ChatGPT is trained on a large dataset of human conversation transcripts and is capable of generating highly engaging and natural-sounding dialogue with humans.
2023	GPT-4, which accepts multi-modal inputs (both text and visual inputs) for generating text, released in March 2023



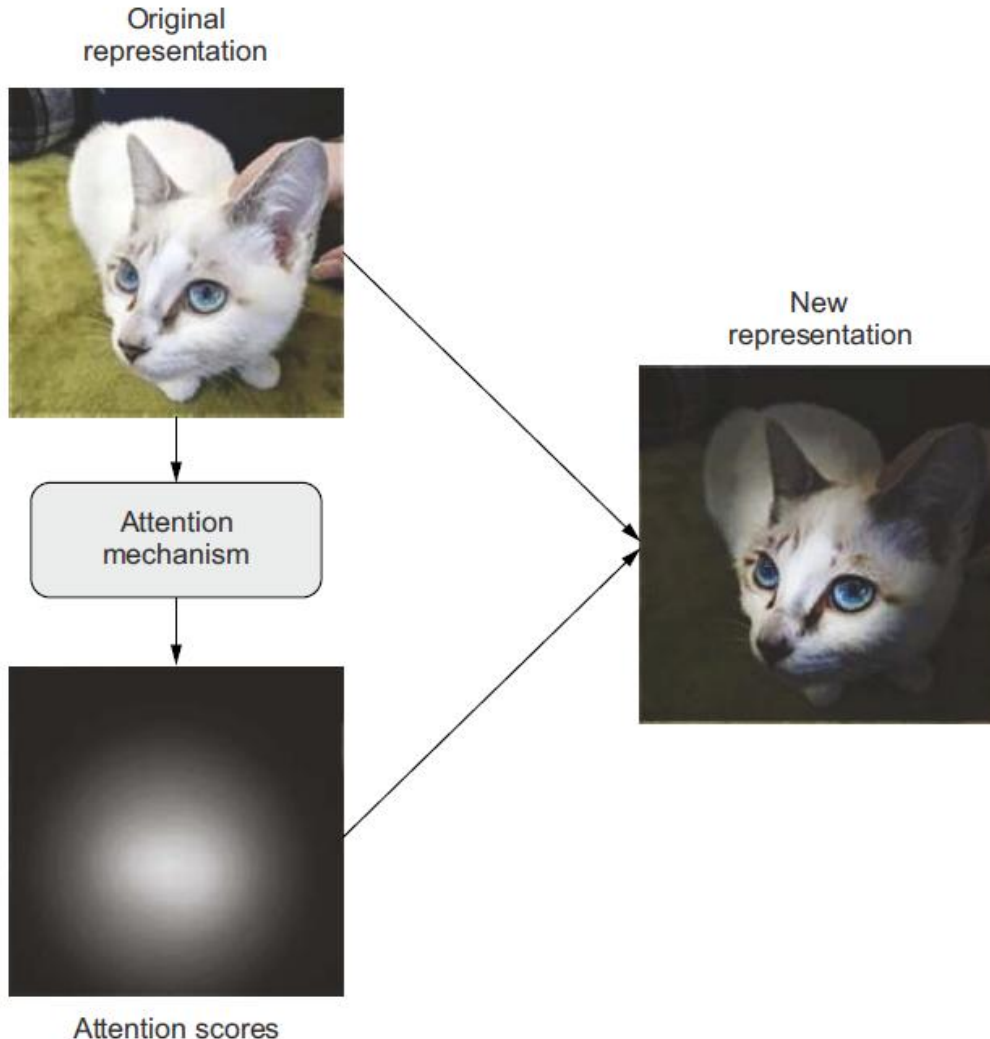
Neural attention

- Transformers essentially leverage on a simple mechanism known as neural attention to build powerful sequence models
- What is neural attention? When we are cramming for exams, do we read every single word in every single textbook with the same care?

Neural attention

- Simple, yet powerful concept – not all the info going into a model is equally important, so the model should pay more attention to some features, and less to others
- Hence, in implementation, we basically start by computing importance scores for a set of features: higher scores for more relevant features, lower scores for less relevant ones

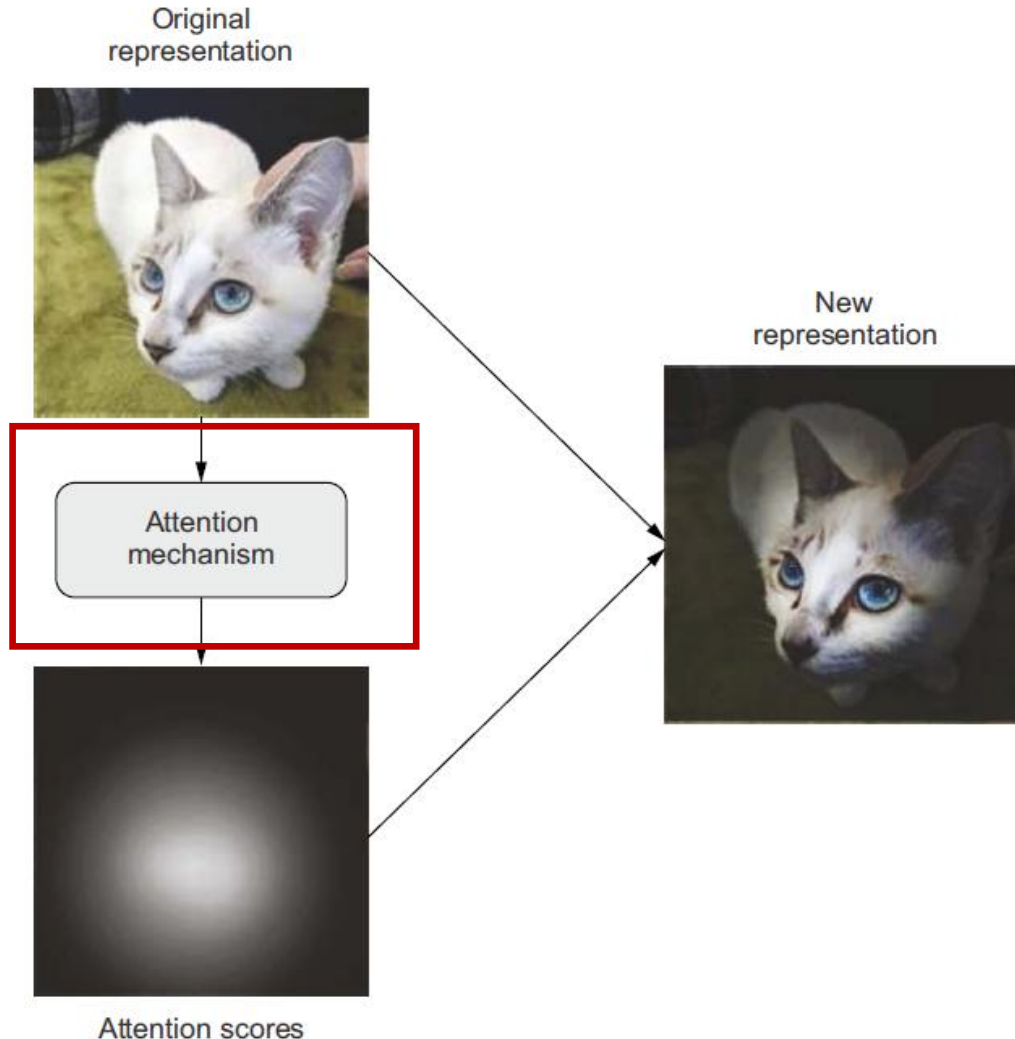
Neural attention



How attention works: Inputs get assigned attention scores, which determines the new representation of the inputs

In doing so, we focus on the more important parts of the features

Neural attention

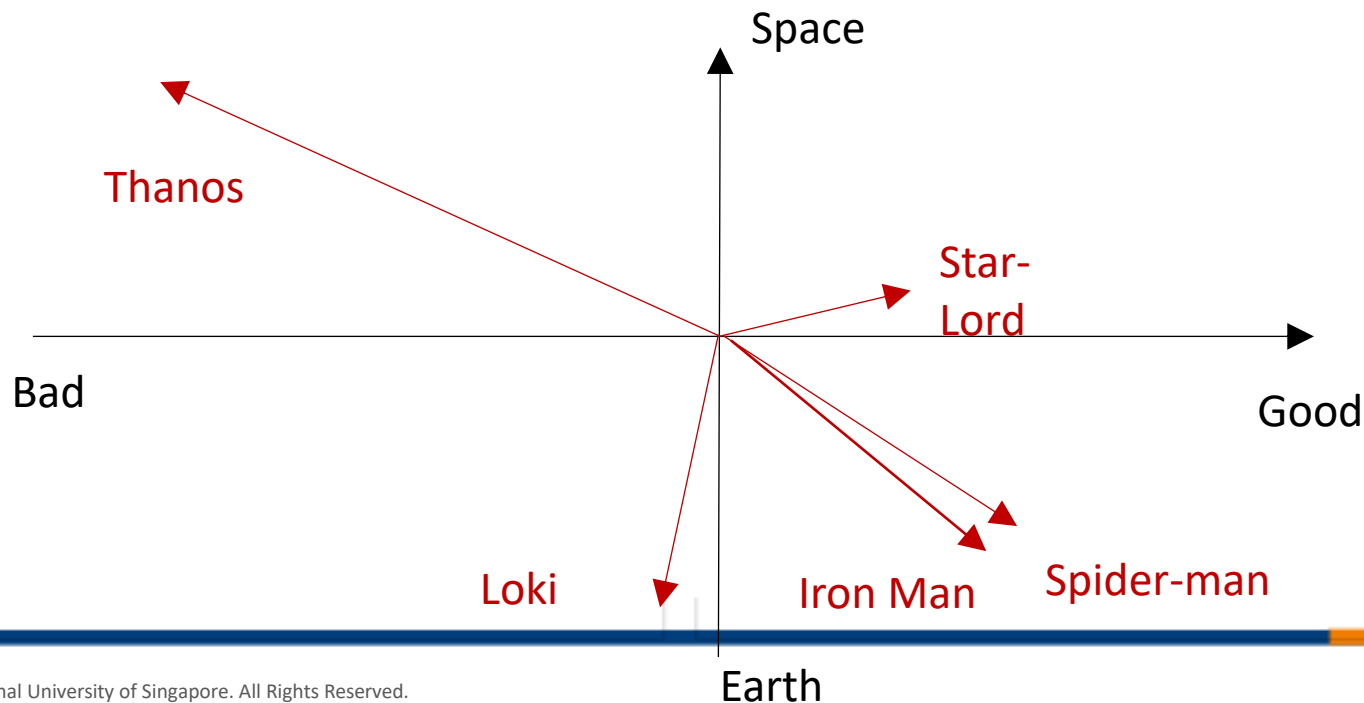


The attention mechanism can do more than just emphasize certain features

It can be used to make features *context-aware*

Self-attention

- We've learned about embeddings previously and how the directions of the vector representations of words tell us the relatedness to other words



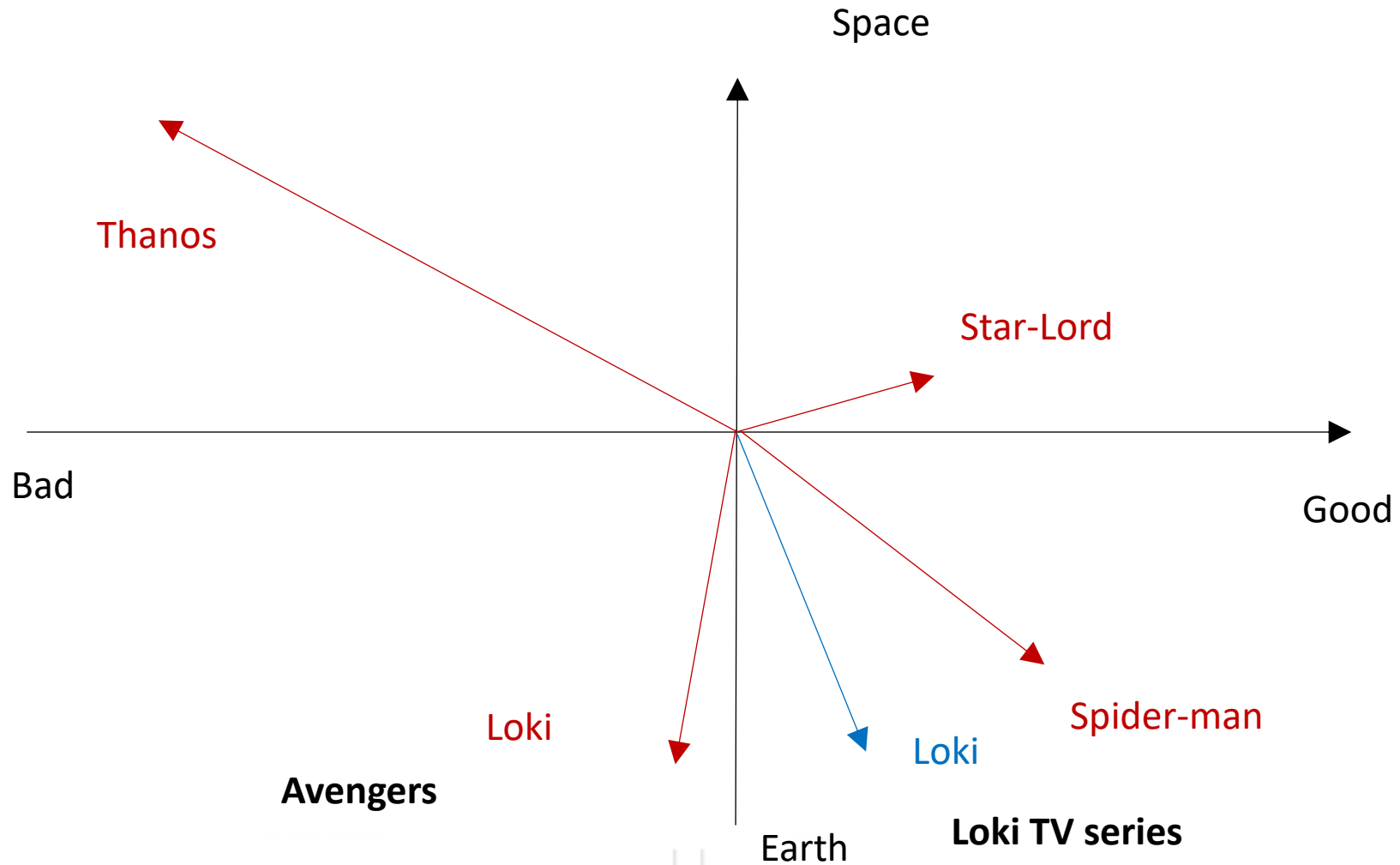
Self-attention

- In an embedding space, a single word has a fixed set of relationships with each and every other word
- But the meaning of a word is context-specific, and therefore relationships to other words can change
- E.g. the word “watch” has a different meaning in the sentences “I will watch over the baby” and “I bought a new watch today”

Self-attention

- Therefore, a smart embedding space would provide a different vector representation for a word, depending on the other words surrounding it
- If the context changes, then the vector representation should change as well

Self-attention

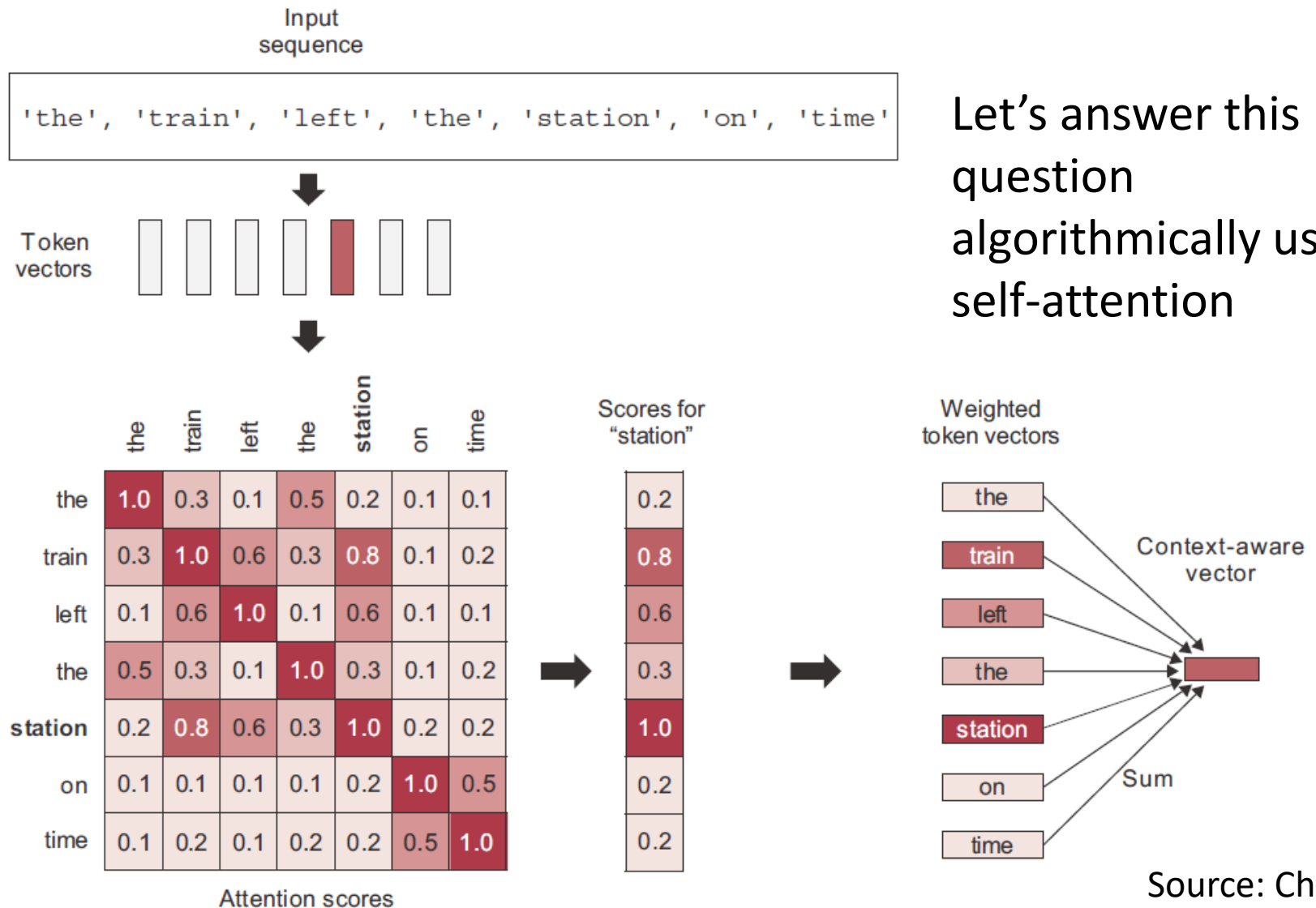


Self-attention

- So that's where **self**-attention comes in, which is to “look unto ourselves”, at say the input sentence itself
- We will then adjust the representation (e.g. embedding) of a word using the representations of other related words in the sentence
- What comes out are context aware token representations (embeddings)

Consider the word “station” in this e.g. What kind of station is it?

Let's answer this question algorithmically using self-attention



Step 1: compute attention scores

- To compute the attention scores, i.e. the relevancy or relatedness scores between any two words in our sentence, we can use the dot product between the two word vectors (that we saw earlier)

	the	train	left	the	station	on	time	
the	1.0	0.3	0.1	0.5	0.2	0.1	0.1	0.2
train	0.3	1.0	0.6	0.3	0.8	0.1	0.2	0.8
left	0.1	0.6	1.0	0.1	0.6	0.1	0.1	0.6
the	0.5	0.3	0.1	1.0	0.3	0.1	0.2	0.3
station	0.2	0.8	0.6	0.3	1.0	0.2	0.2	1.0
on	0.1	0.1	0.1	0.1	0.2	1.0	0.5	0.2
time	0.1	0.2	0.1	0.2	0.2	0.5	1.0	0.2

Attention scores

Scores for "station"

Step 1: compute attention scores

- We notice that the scores go from 0 to 1, and that's because usually some scaling and normalisation is also done, e.g. softmax

	the	train	left	the	station	on	time	
the	1.0	0.3	0.1	0.5	0.2	0.1	0.1	Scores for "station" 0.2 0.8 0.6 0.3 1.0 0.2 0.2
train	0.3	1.0	0.6	0.3	0.8	0.1	0.2	
left	0.1	0.6	1.0	0.1	0.6	0.1	0.1	
the	0.5	0.3	0.1	1.0	0.3	0.1	0.2	
station	0.2	0.8	0.6	0.3	1.0	0.2	0.2	
on	0.1	0.1	0.1	0.1	0.2	1.0	0.5	
time	0.1	0.2	0.1	0.2	0.2	0.5	1.0	
Attention scores								

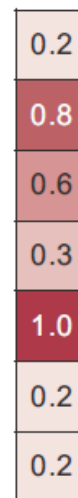


Step 2: Compute sum of all word vectors, weighted by attn scores

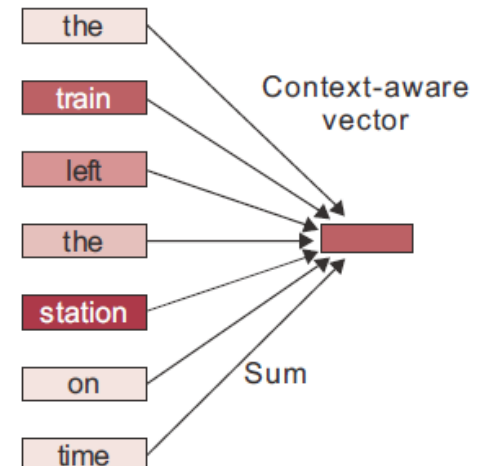
- For “station”, we are going to get a set of attention scores for how related each other word is to it
- Now, we use these scores to weight the respective word vectors

- Finally, we will sum up these weighted word vectors (essentially getting a weighted average) to yield the new embedding for “station”

Scores for
“station”

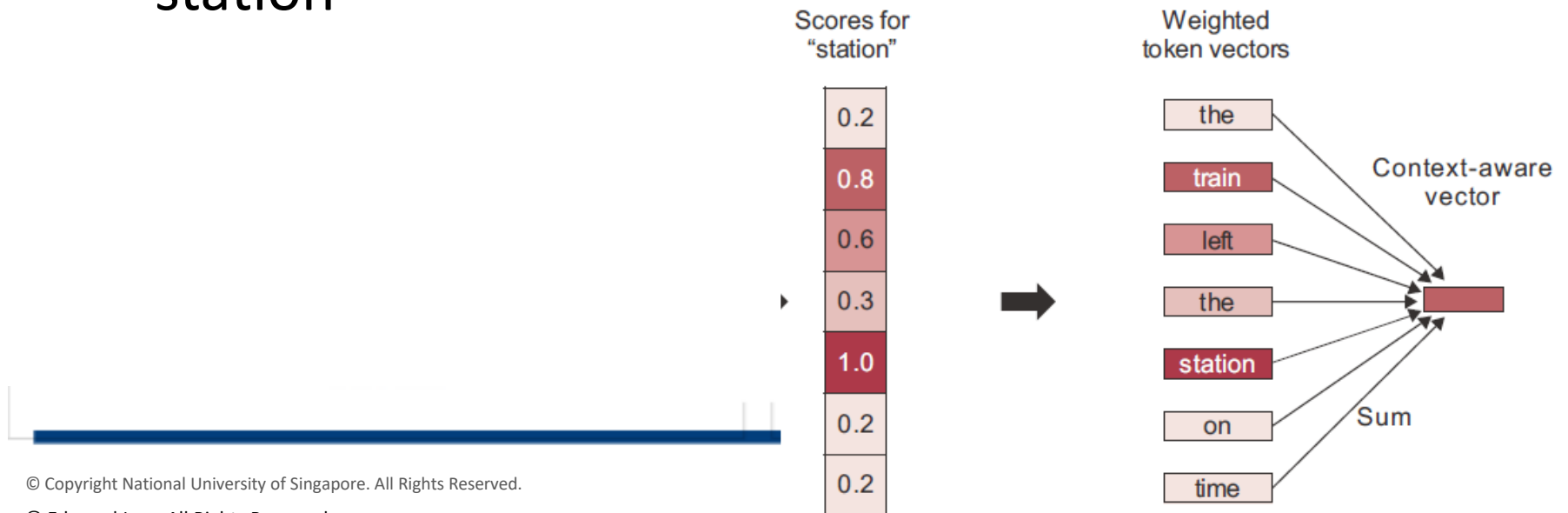


Weighted
token vectors



Step 2: Compute sum of all word vectors, weighted by attn scores

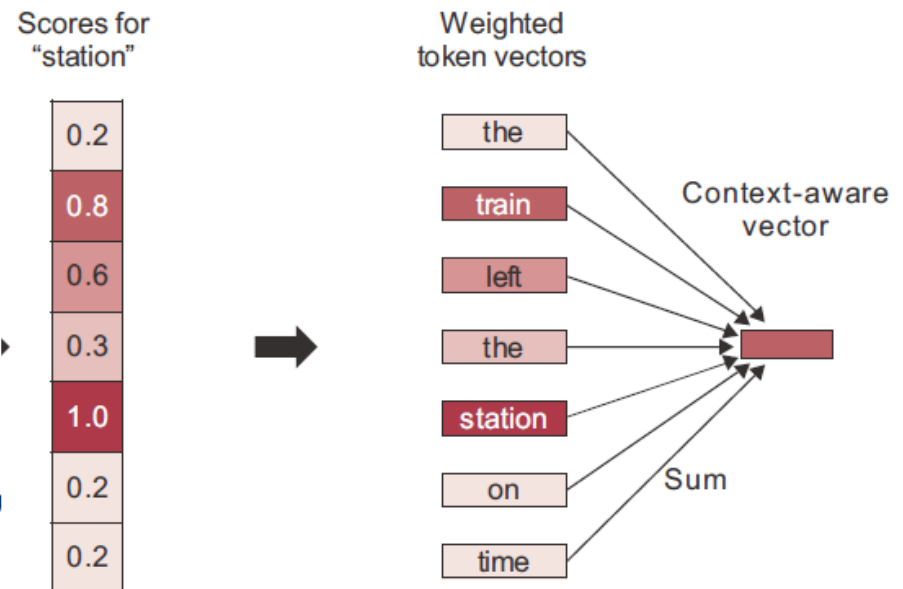
- Words closely related to “station”, including “station” itself, will have higher attention scores and therefore contribute more to the sum
- The resulting vector is our new representation for “station”



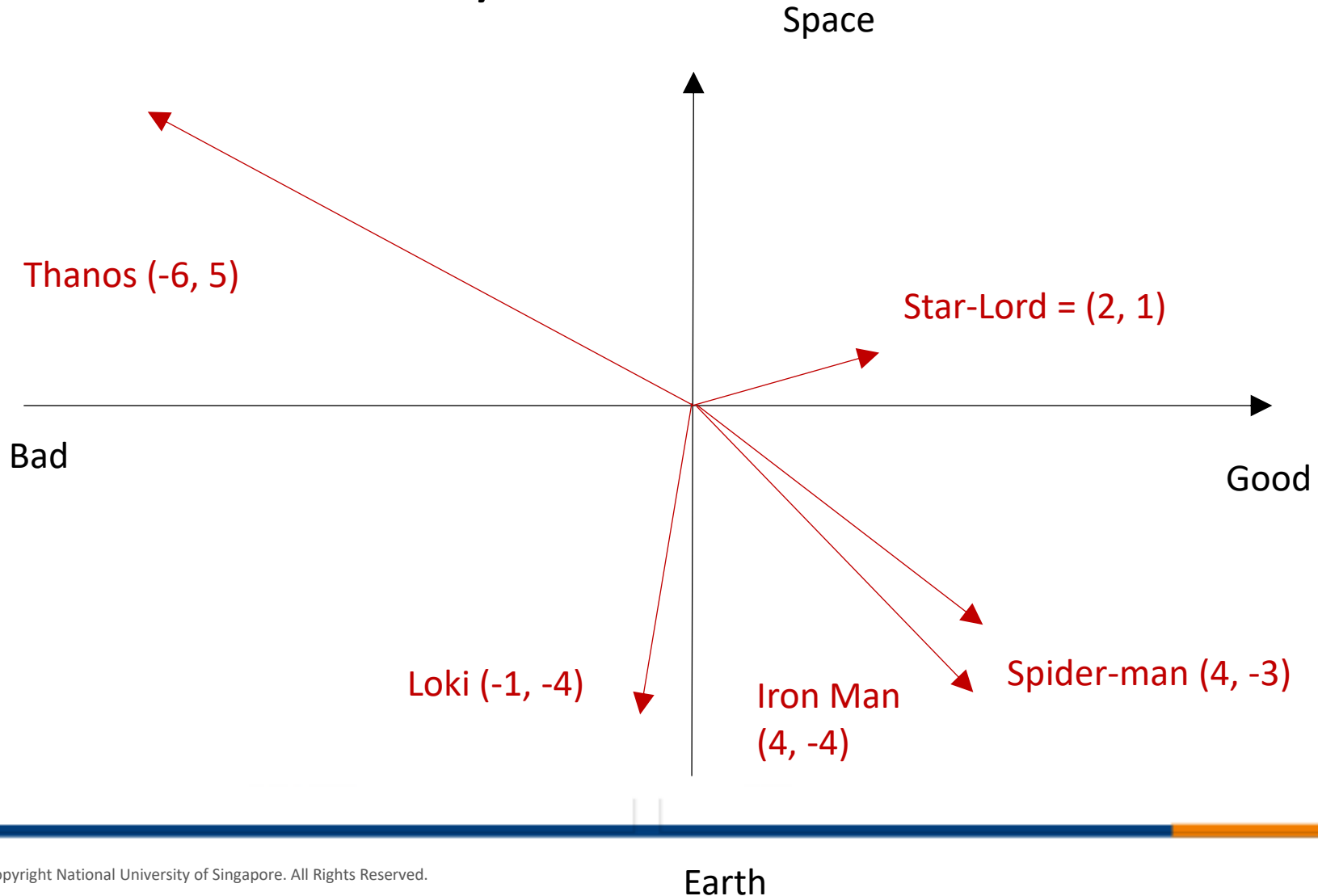
Step 2: Compute sum of all word vectors, weighted by attn scores

- This new embedding has incorporated the surrounding context; for instance, we see that it includes a fair bit of the embedding for “train”, given the high attention score of 0.8

- This therefore clarifies that we are indeed talking about a train station



Class activity



Class activity – Table 1

- Refer to activity Excel sheet

	Thanos	Star-Lord	Iron-Man	Spider-man	Loki
Thanos					
Star-Lord					
Iron-Man					
Spider-man					
Loki					

Class activity

- Table #2

	Thanos	Star-Lord	Iron-Man	Spider-man	Loki
Thanos					
Star-Lord					
Iron-Man					
Spider-man					
Loki					

Class activity

- Table 3: Dividing Table #1 with Table #2

	Thanos	Star-Lord	Iron-Man	Spider-man	Loki
Thanos					
Star-Lord					
Iron-Man					
Spider-man					
Loki					

Class activity

- Table 4: Scaling Table #3

	Thanos	Star-Lord	Iron-Man	Spider-man	Loki
Thanos					
Star-Lord					
Iron-Man					
Spider-man					
Loki					

Class activity

- Finally, let's apply our attention scores to the existing embedding for Thanos, to get context-aware embedding for Thanos!

Self-attention

- Instead of treating each word embedding as is, in a fixed embedding space
- We are now revising the embeddings by being “attentive” to the surrounding embeddings, and incorporating these surrounding embeddings directly into the new embedding → self-attention!

The Query-Key-Value model

- The self-attention mechanism that we have just described can be schematically represented as:
- $\text{outputs} = \text{sum}(\text{inputs} * \text{pairwise_scores}(\text{inputs}, \text{inputs}))$

↑
C

↑
A

↑
B

- To recap, for every token in inputs (A), calculate the attention scores with every token in inputs (B), and use these scores to weight a sum of tokens from inputs (C)

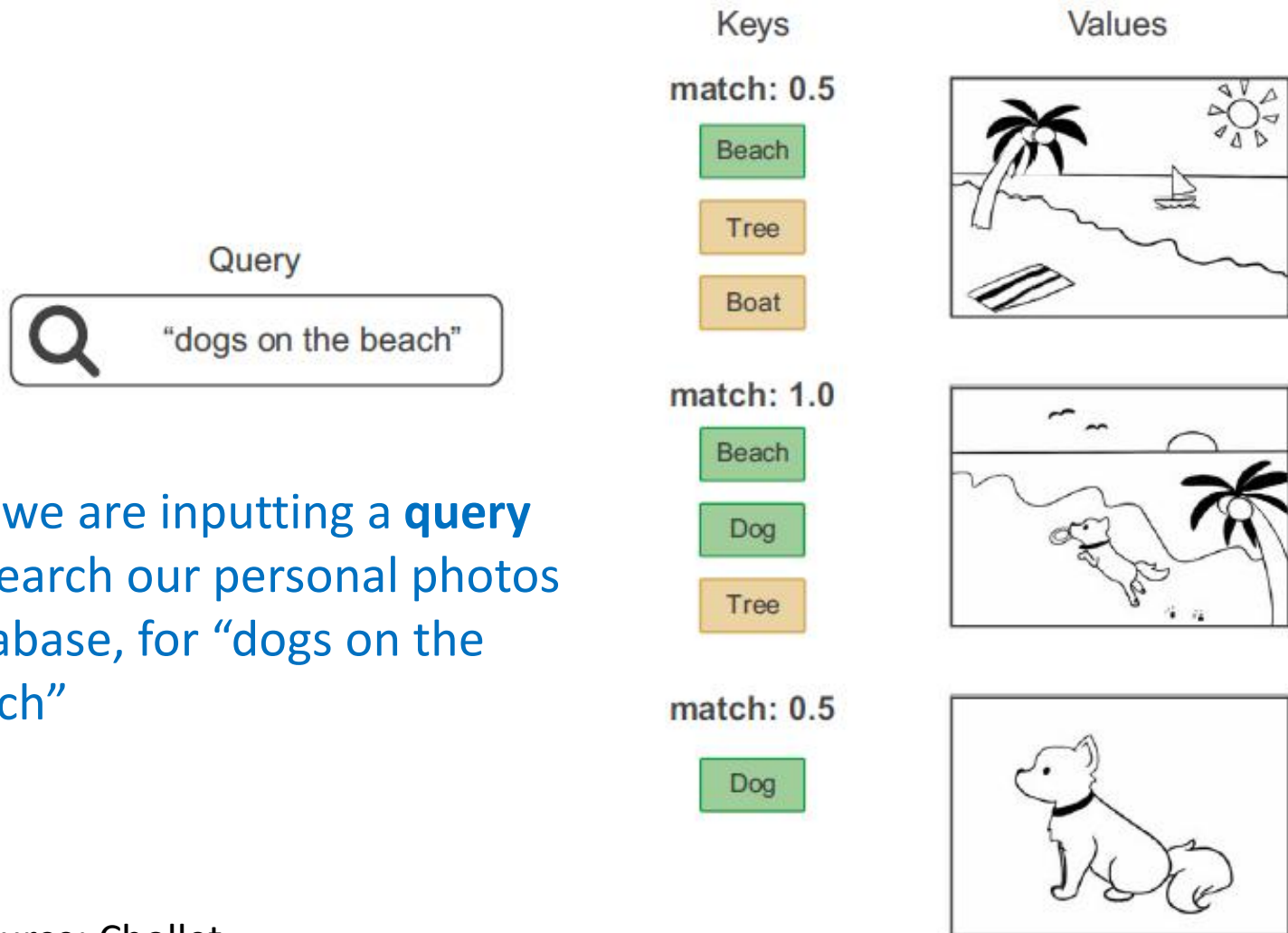
The Query-Key-Value model

- In our example, (A), (B) and (C) all refer to the same sentence, hence the term *self-attention*
- But they need not necessarily refer to the same input sequence at all; when the inputs are different, this is more generally termed *attention*
- Let's re-write the schematic we have seen using the typical Transformer terminology

The Query-Key-Value model

- $\text{outputs} = \text{sum}(\text{values} * \text{pairwise_scores}(\text{query}, \text{keys}))$
- Hence, for each element in query, compute how related the element is to every key, and use these scores to weight a sum of values
- Let's use an example to illustrate when attention is applied to three different inputs

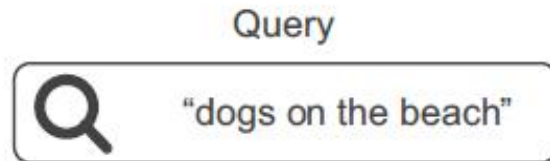
The Query-Key-Value model



Say we are inputting a **query** to search our personal photos database, for "dogs on the beach"

The Query-Key-Value model

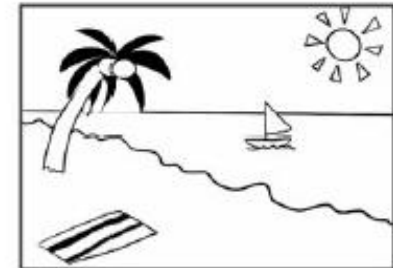
We might have tagged each of our photos previously with keywords. These are the **keys**



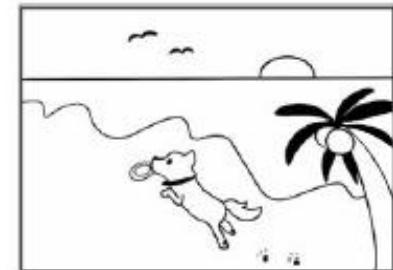
Keys
match: 0.5



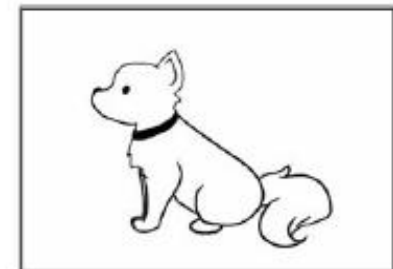
Values



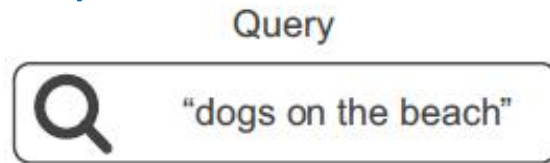
match: 1.0



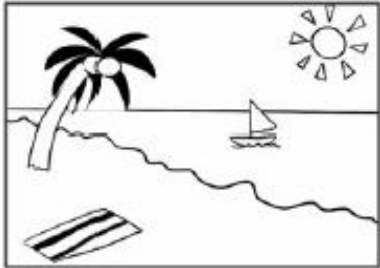
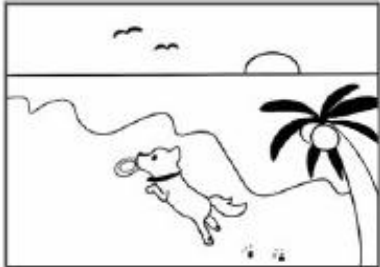
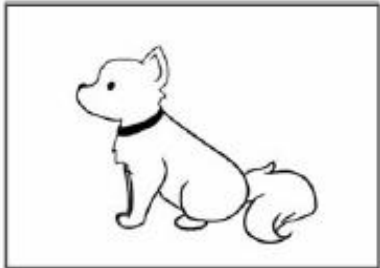
match: 0.5



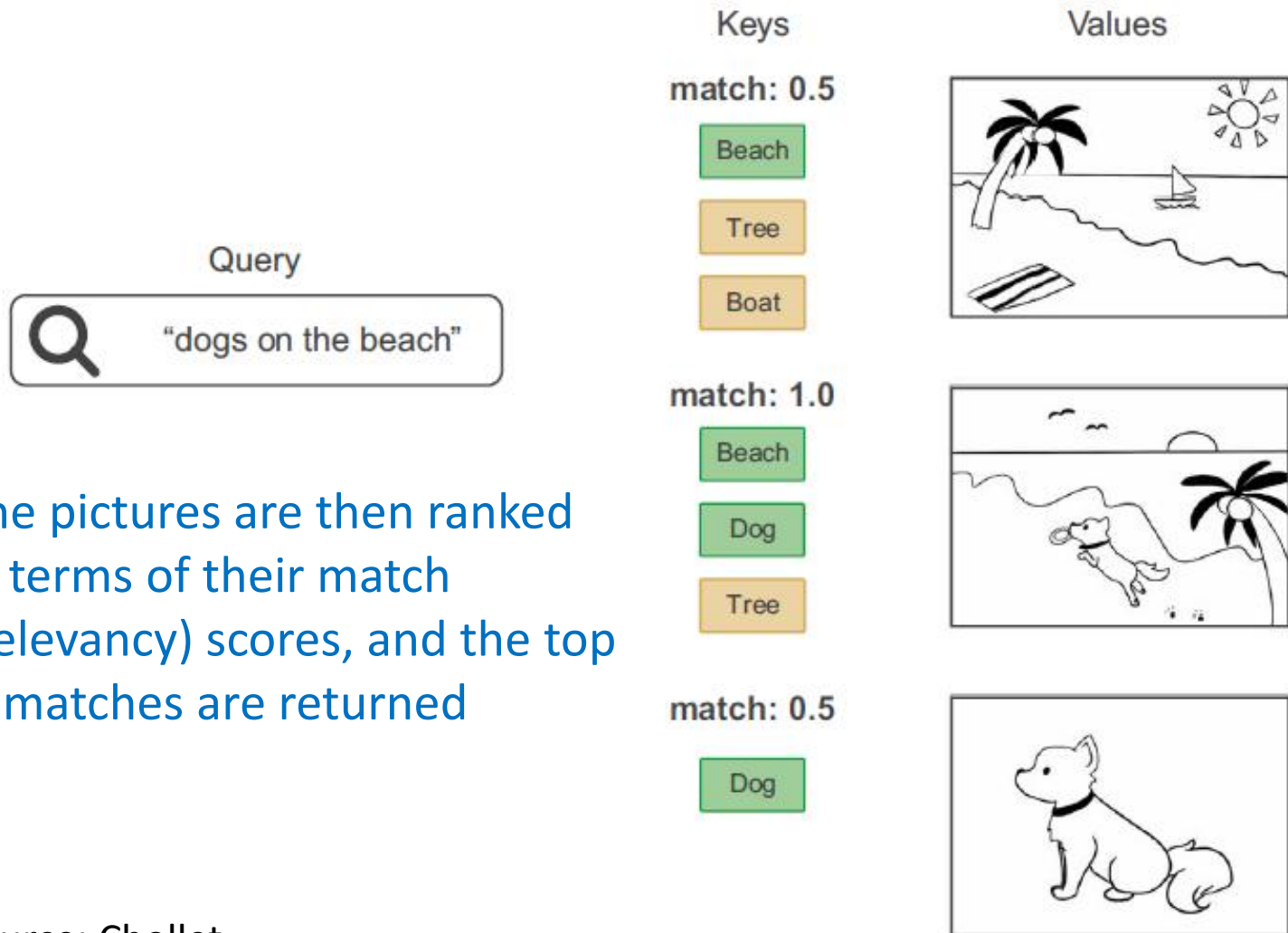
The search engine will then compare our query with our keys, and compute a match (or relevancy score), i.e. the pairwise score between our query and key. The more keys match a query, the higher the relevancy score



For example, the second photo has a higher match score compared with the others, since two keywords match the query

Keys	Values
match: 0.5 Beach Tree Boat	
match: 1.0 Beach Dog Tree	
match: 0.5 Dog	

The Query-Key-Value model



The pictures are then ranked in terms of their match (relevancy) scores, and the top N matches are returned

The Query-Key-Value model

- In essence, this describes the Transformer-style attention we have been discussing so far
- We have an input that describes something we're looking for → query
- We've got a body of knowledge that we'll be extracting info from → values (our photos in this case)
- Each value (photo) is assigned a key that describes the value in a format that can be easily compared to our query

The Query-Key-Value model

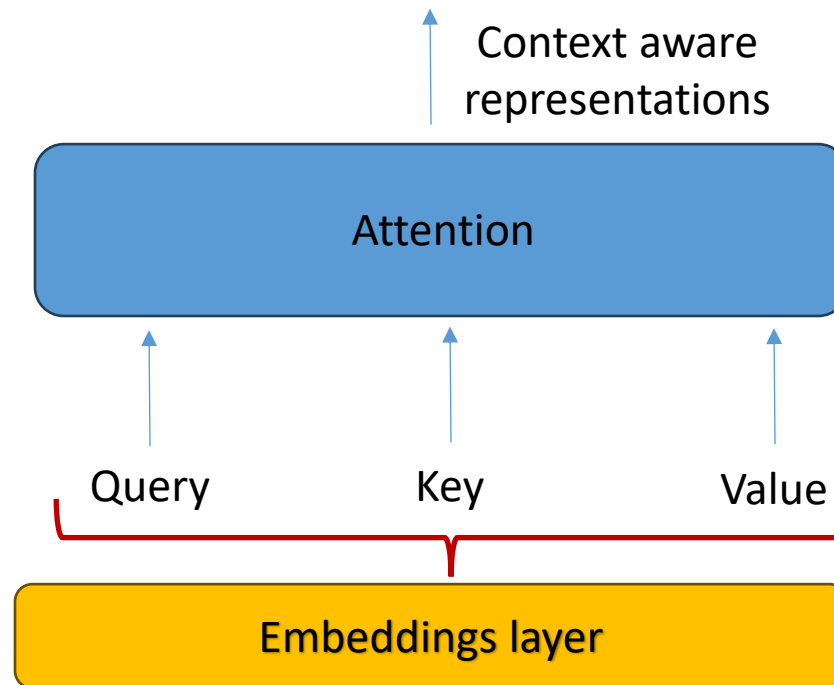
- In our current example, we are returning photos from our database (**weighted sum of values**) that have a high matching score (**pairwise scores**) from matching the tags (**keys**) to the search terms (**query**)
- Based on the pairwise relevancy scores between matching the keys to the query, we will have the following “weighted sum of values”:
 - $0.5 * \text{Photo1} + 1.0 * \text{Photo2} + 0.5 * \text{Photo3}$

The Query-Key-Value model

- Which means that photo 2 will be returned as a first priority since its weight is the highest, followed by photos 1 and 3
- In our previous example, we are returning new word embeddings (**weighted sum of values**) for our sentence, after considering how each word in our sentence (**query**) is related to every other word in our same sentence (**key, via pairwise attention scores**)

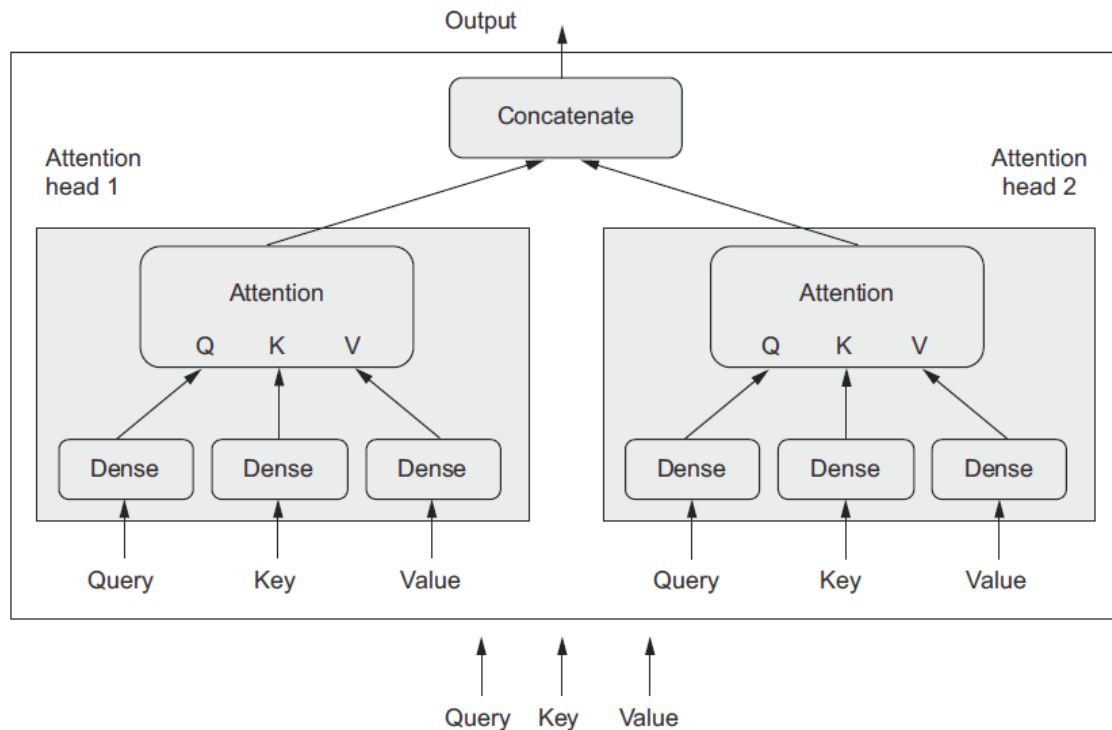
Big picture

- So that's how neural attention works; let's take a look now at how it is implemented in a Transformer in a NN!
- Fundamentally, we just need the following



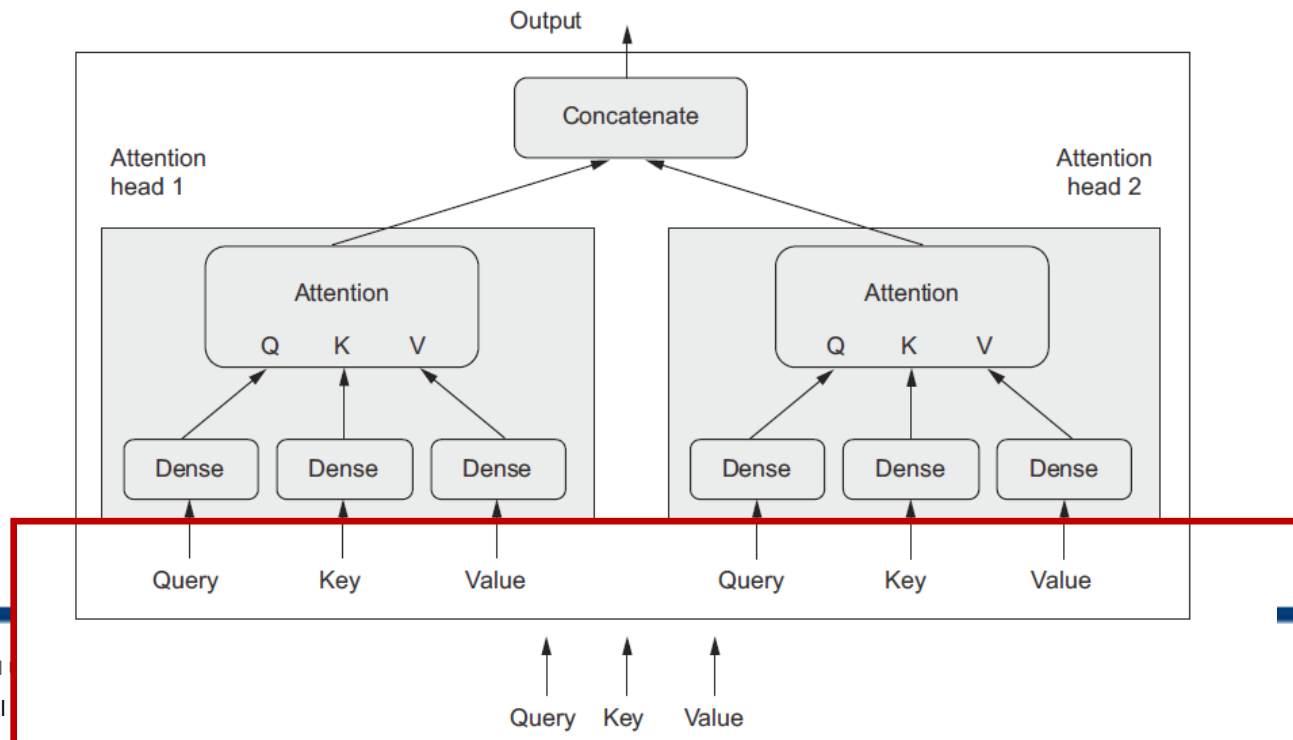
Multi-head attention

- “Multi-head attention” is a tweak to the self-attention mechanism; we will use the following figure to illustrate



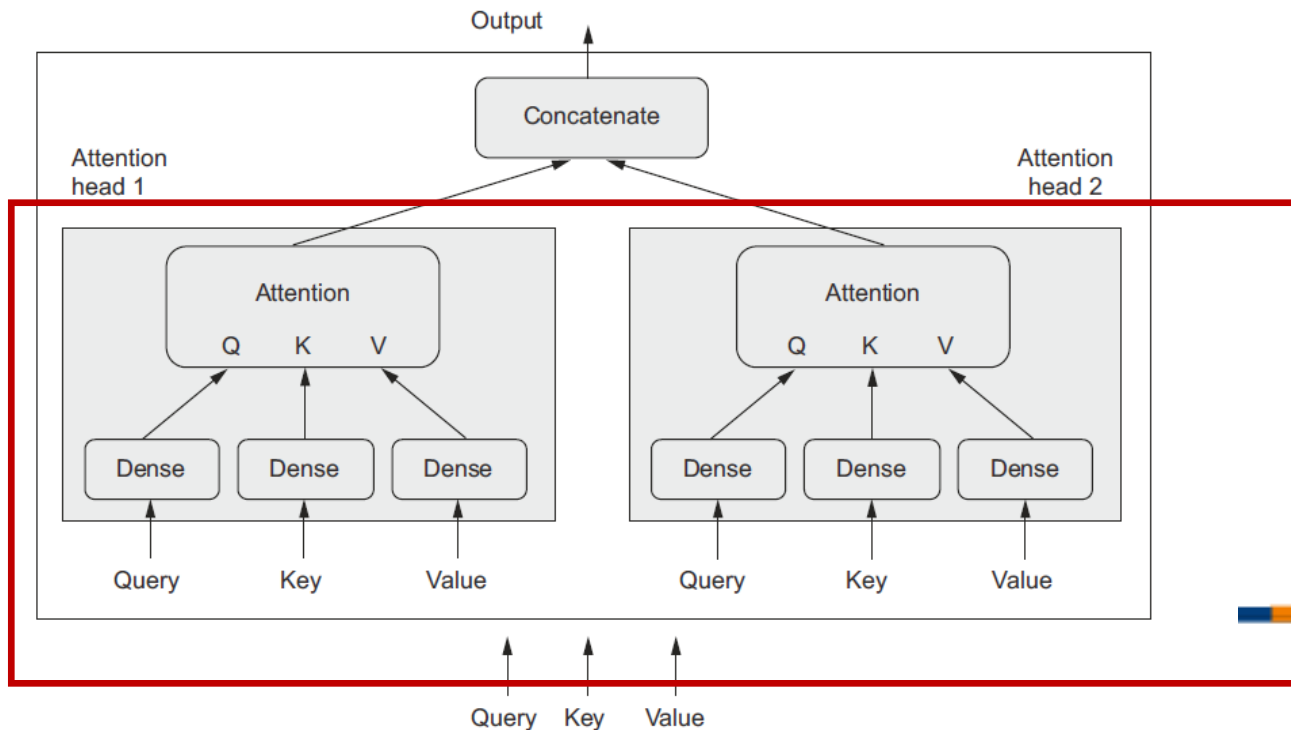
Multi-head attention

- The initial query, key and value (in our example, they are all the same and refer to the word embeddings in our sentence) are split across multiple heads, in this case 2 heads



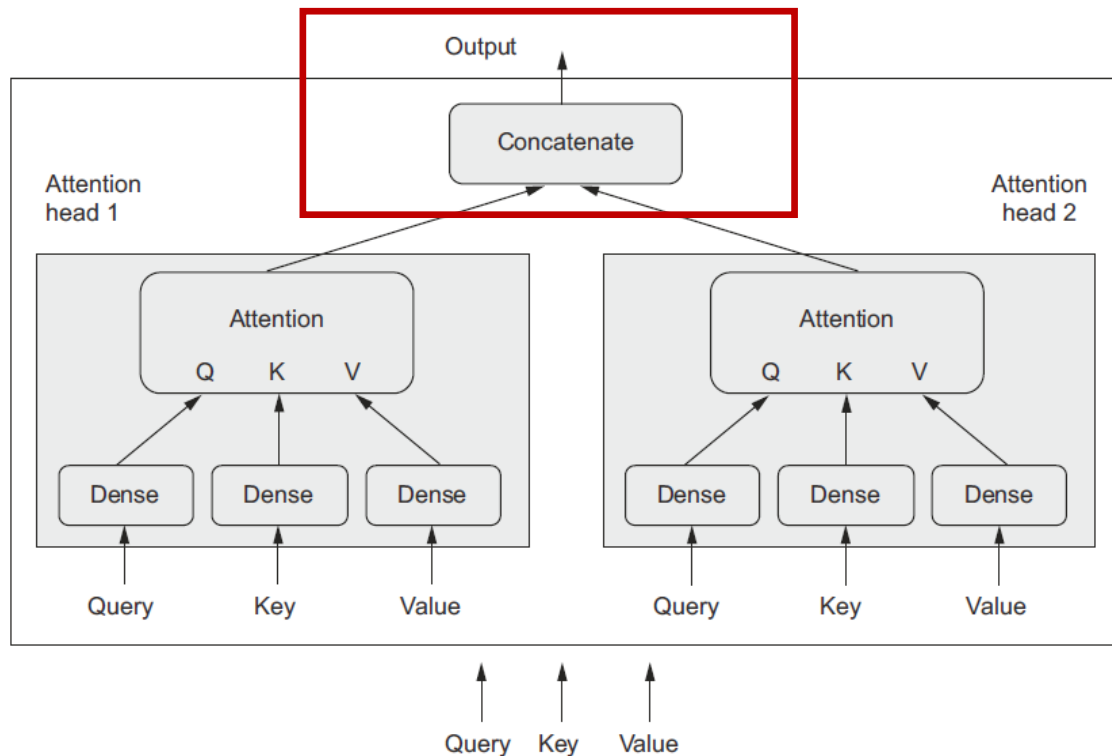
Multi-head attention

- In each head, the query, key and value are passed through three independent sets of dense layers, yielding three vectors which are then processed via neural attention, per what we have discussed



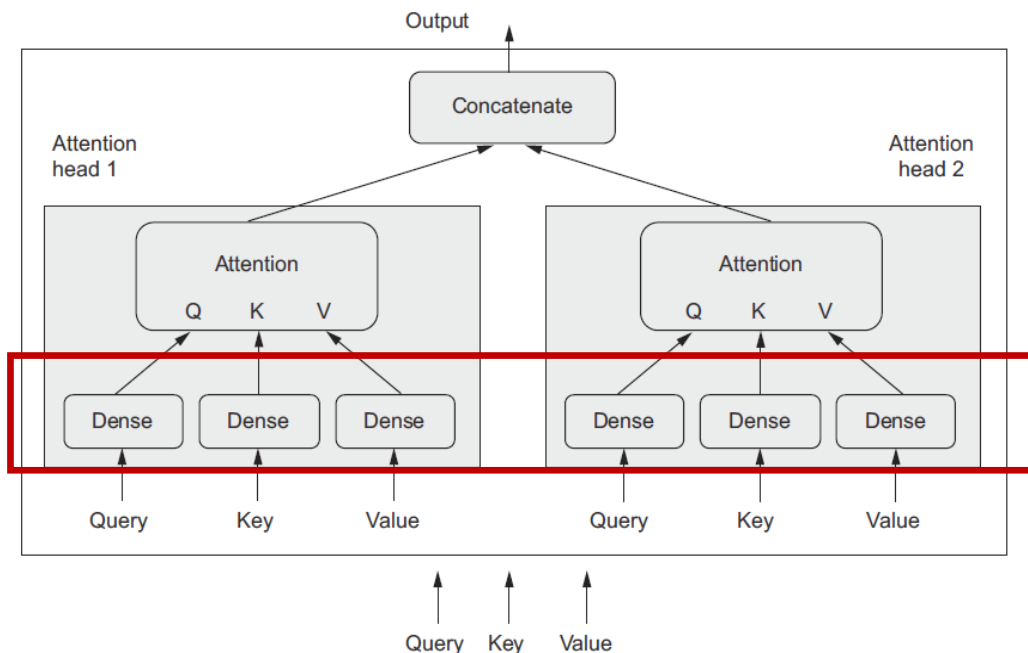
Multi-head attention

- At the end, the output from each head are then further concatenated to yield the overall output



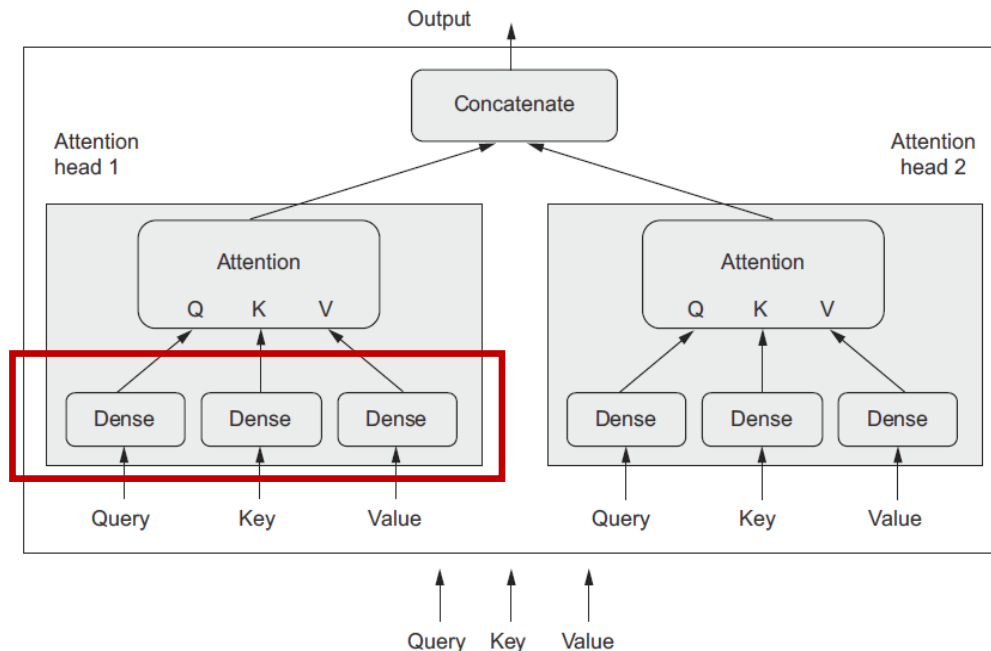
Multi-head attention

- The rationale for having the dense layers act on the query, key and value embeddings is so that the Transformer layer can actually learn something
- Rather than just being purely a stateless transformation that would require layers before or after it to be useful



Multi-head attention

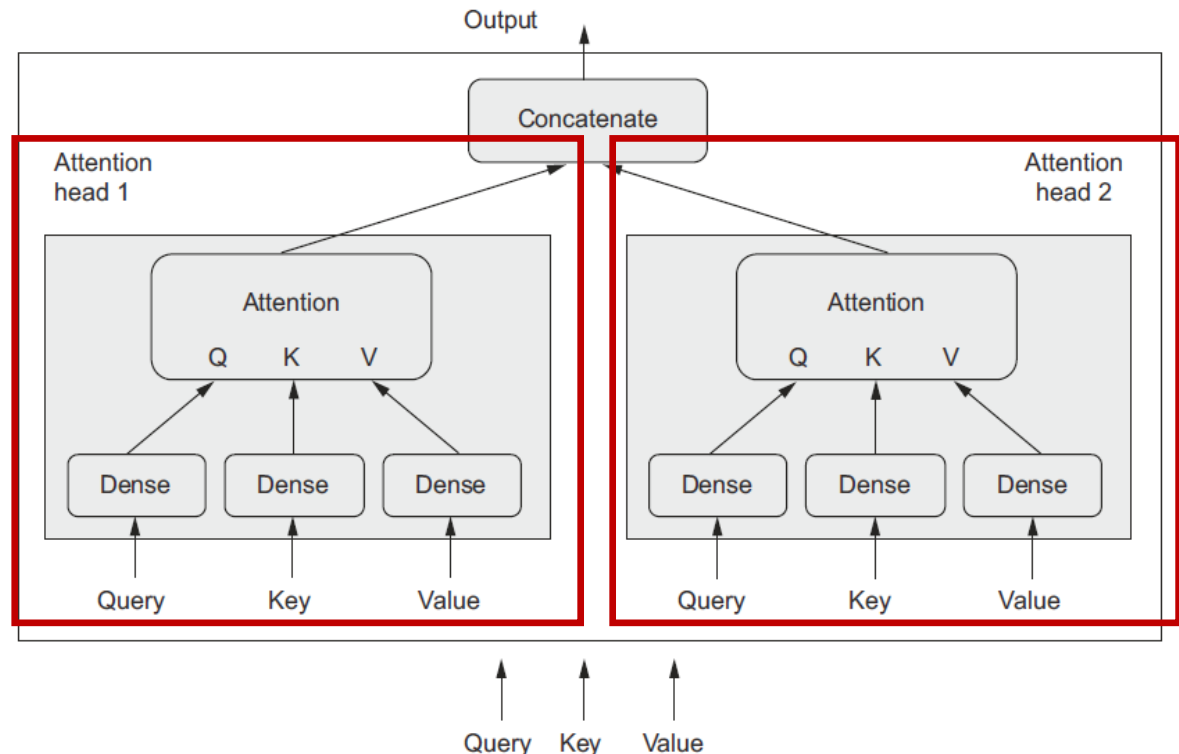
- So, essentially three independent transformations (as circled) are applied to each word embedding to generate the query, key and value vectors
- These transformations have their own set of trainable parameters, which allows the self-attention layer to focus on different semantic aspects of the sequence



Multi-head attention

- Having independent heads also allow us to attend to different aspects of the sequence at once

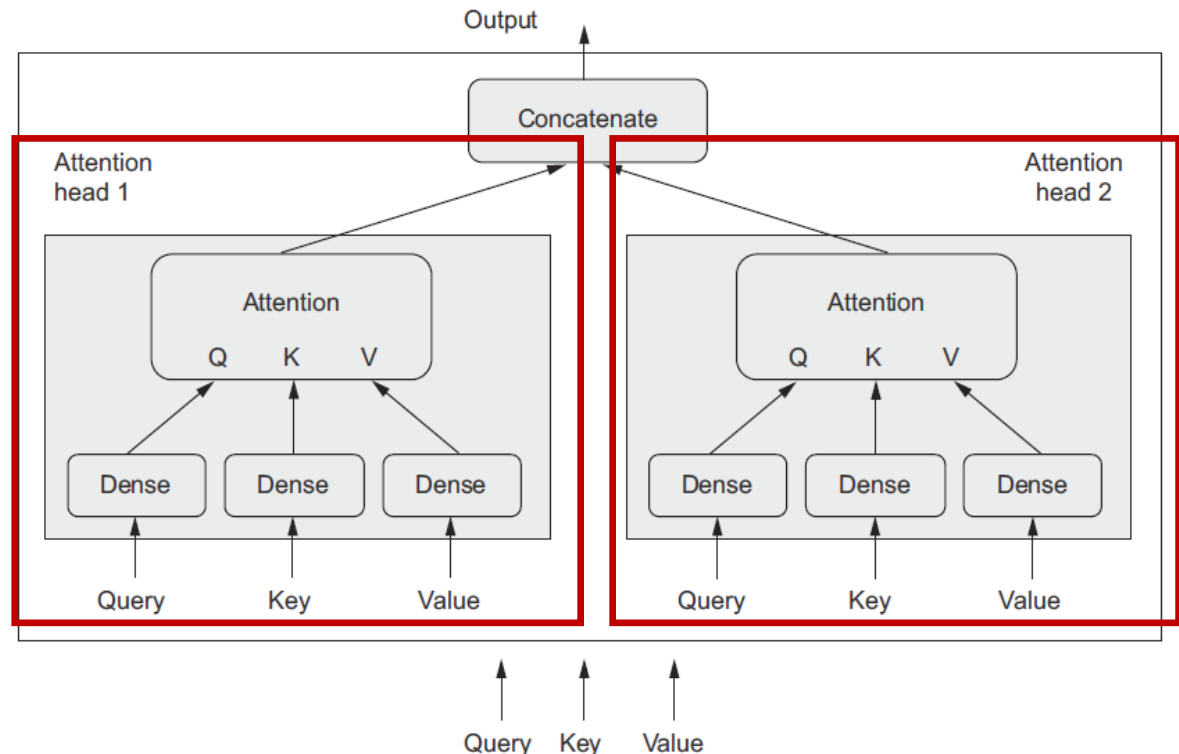
Each attention head tends to focus largely on one aspect, e.g. subject-verb interaction, neighbouring adjectives



Multi-head attention

- This multi-head attention then serves as a layer as part of the overall Transformer in a neural network model

The concatenated output from the attention heads is also fed through a final layer to produce an output tensor of shape that is suitable for the dense layers downstream

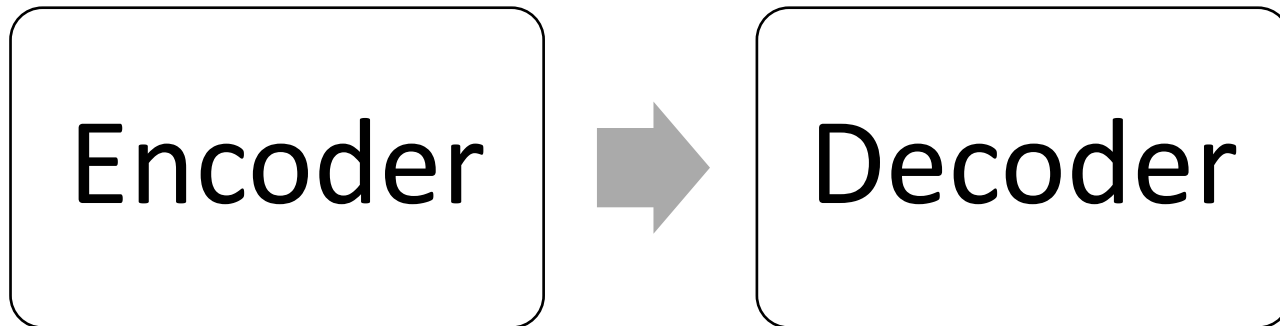


The Transformer architecture

- If adding extra dense layers is so useful in helping us to learn something about our inputs, why don't we also apply a couple to the output of the attention mechanism?
- And in fact, this is the case for the actual Transformer architecture

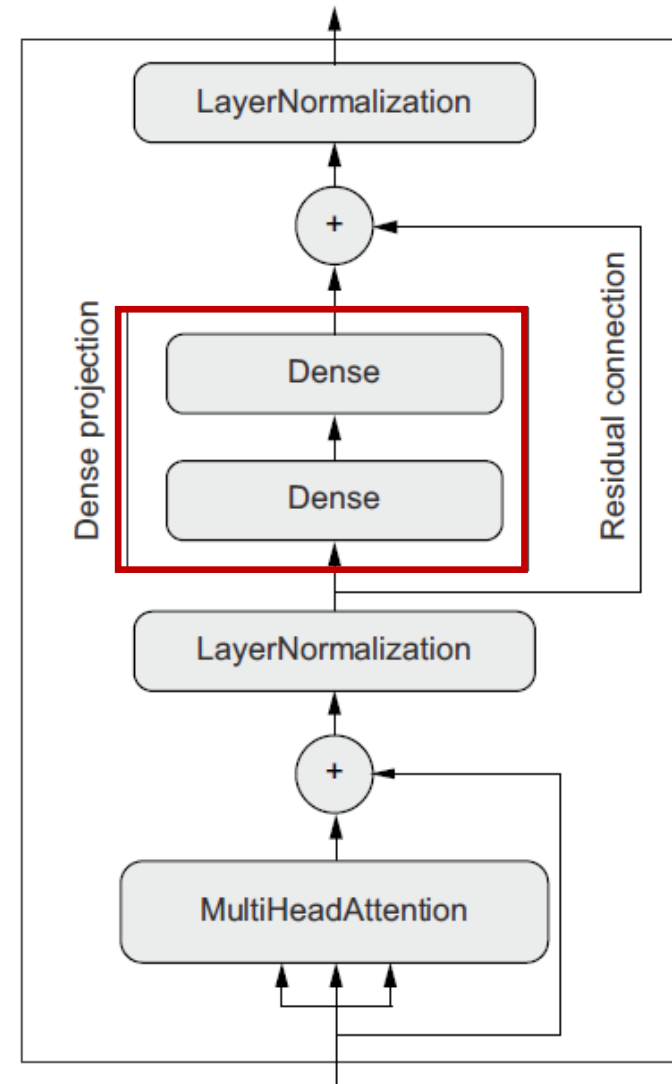
Transformer architecture

- The overall transformer architecture comprises an encoder and a decoder that works together, or independently



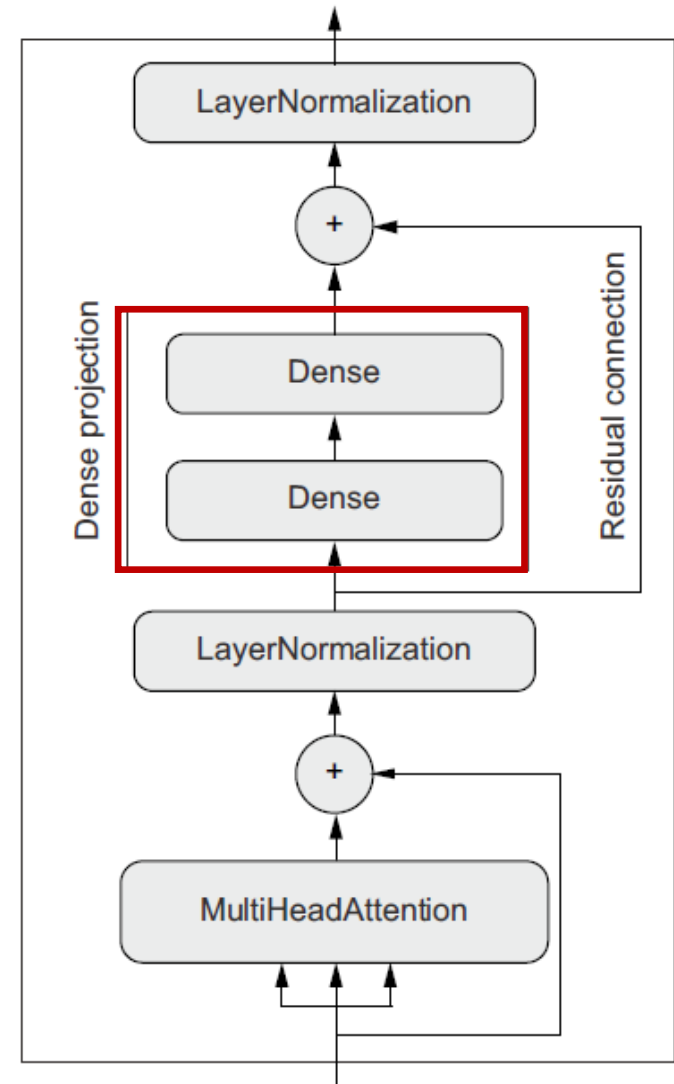
The Transformer architecture

- We will look at the decoder in a bit
- For now, this is what the encoder looks like when we “pop open the hood”, with the two dense layers highlighted, acting on the output from the Multi-Head Attention layer



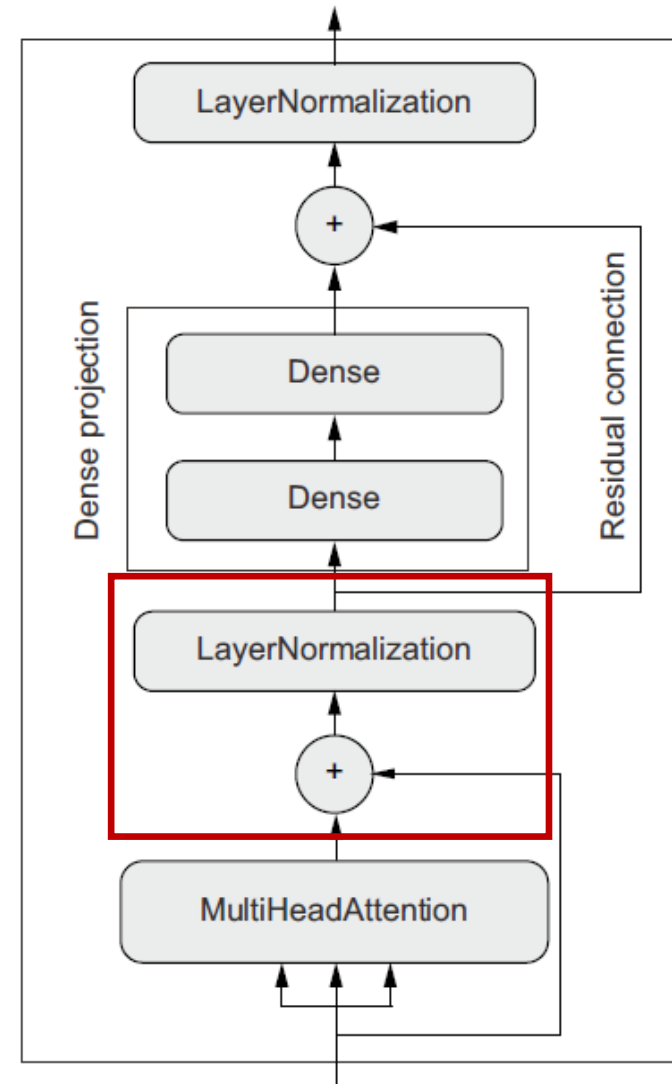
The Transformer architecture

- The dense layers are a simple two-layer fully connected neural network

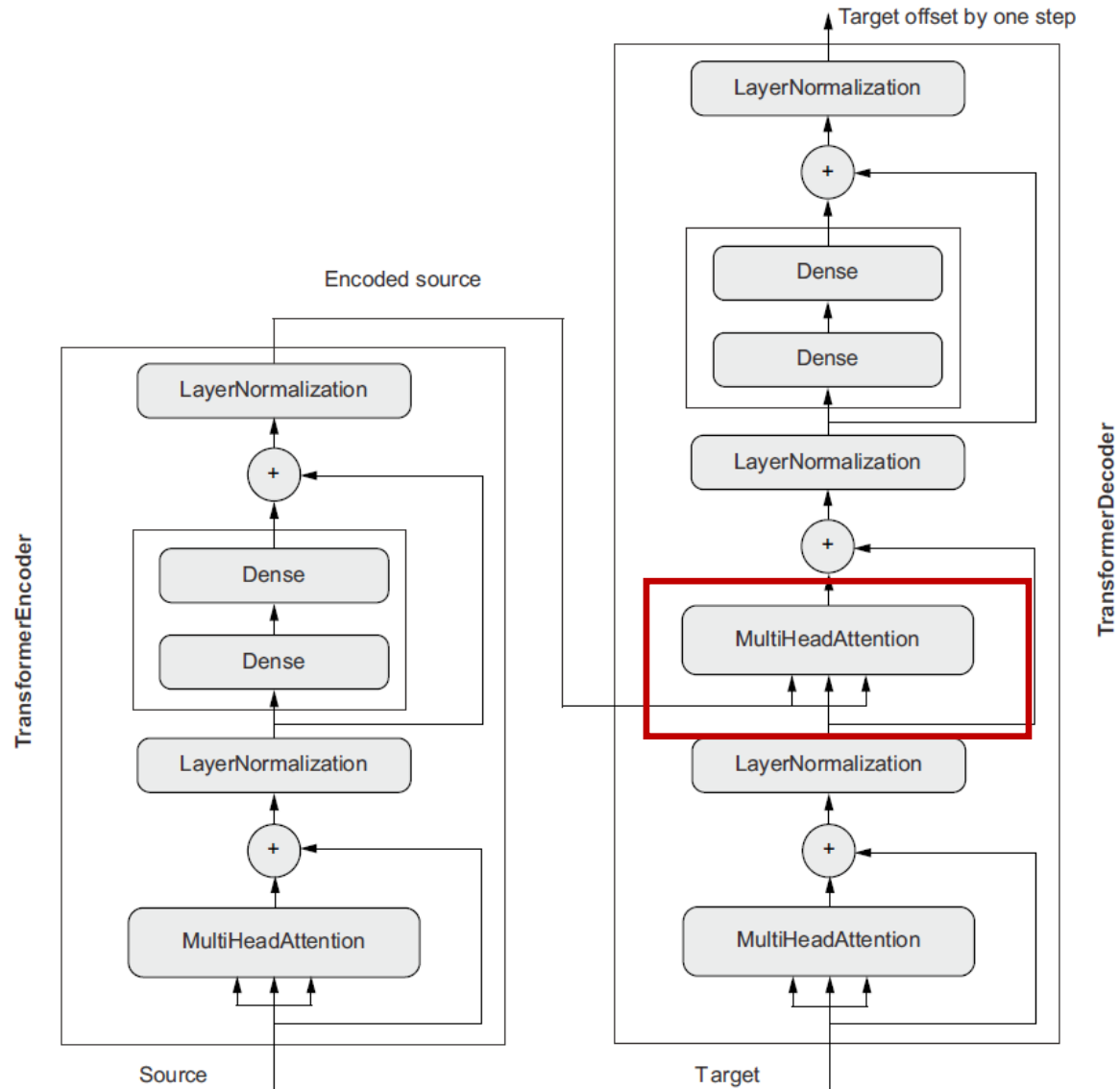


The Transformer architecture

- We also notice we have some residual connections and normalization layers
- They are there to improve the performance of the encoder



The Transformer architecture



The Transformer architecture

- We now put the architecture of the Transformer decoder, alongside that of the encoder
- What's the difference between the two?

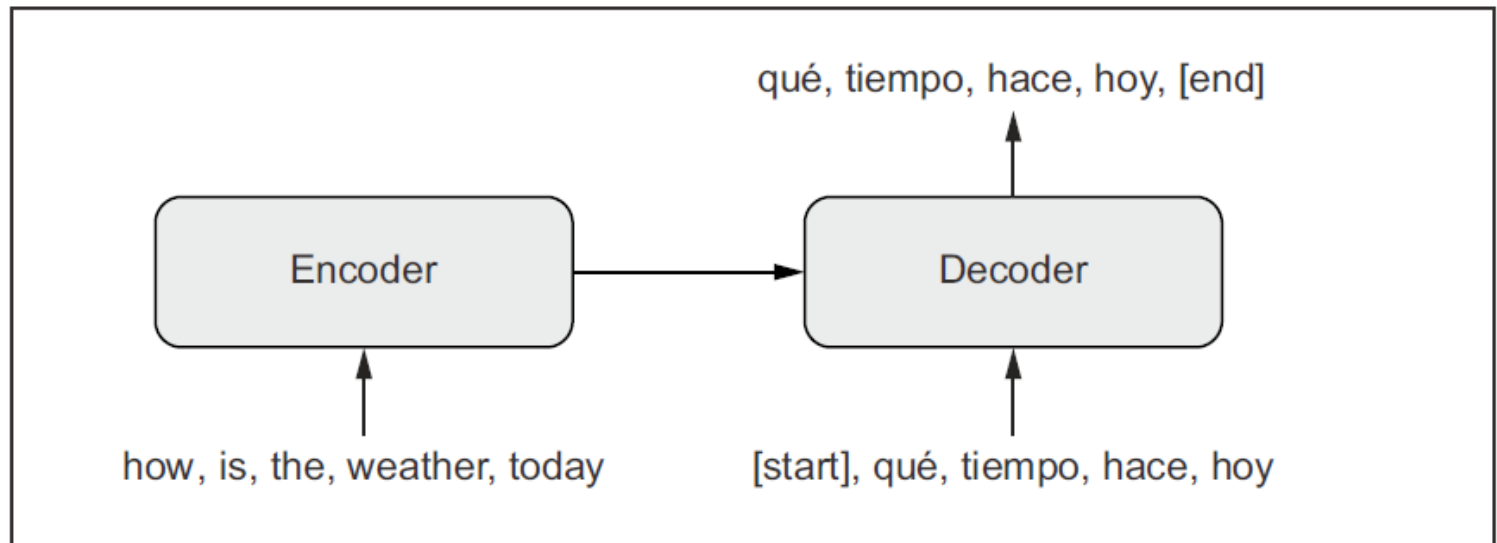
The Transformer architecture

- Now, what does the encoder and decoder do?
- In machine translation (a common application of NLP models), the encoder will process the source sequence (say a sentence in English)
- While the decoder will use the processed source sequence from the encoder, to generate a translated version (say in Spanish)

Encoder + decoder

- When training the model for translation, the encoder will read the source (English) sentence (“How is the weather today”), and encode it into context-aware representations of each token
- i.e. we get a sequence of context-aware embedding vectors

Training phase



Encoder + decoder

- The decoder will then be trained to predict the next token in the target (Spanish) sequence, by looking at both the previous tokens and the encoded source sequence
- For example, predicting “tiempo” as the next word, by looking at the previous token(s) so far “que”, as well as the source English sentence “How is the weather today”

Encoder + decoder

- While doing this, the decoder will leverage on neural attention to determine which tokens from the encoded source sentence are most closely related to the target token it's trying to predict
- For instance, “tiempo” and “weather” should have a strong match

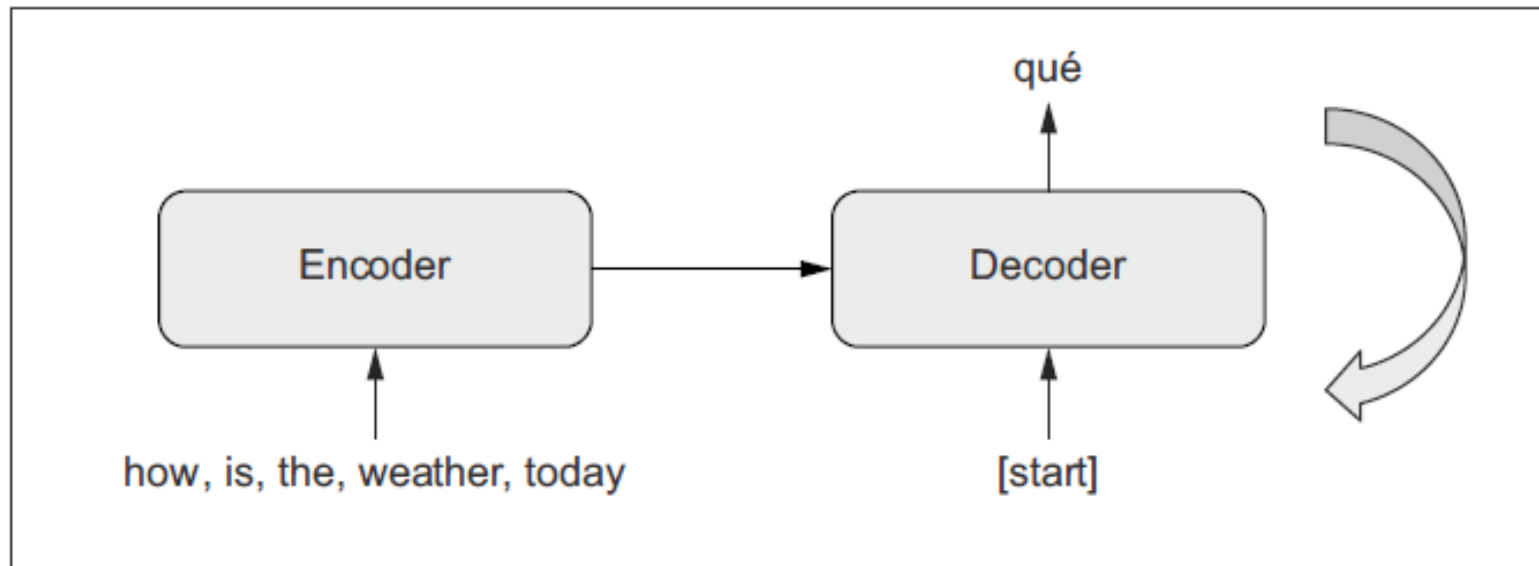
Encoder + decoder

- This exactly comports with the query-key-value model we talked about earlier
- In the decoder, the target (Spanish) sentence is the “query” that we use to pay closer “attention” to different parts of the source (English) sentence (which assumes both the “key” and “value” roles here)

Encoder + decoder

- Once trained, we can use the model to infer a translated (Spanish) sentence, from a source (English) sentence

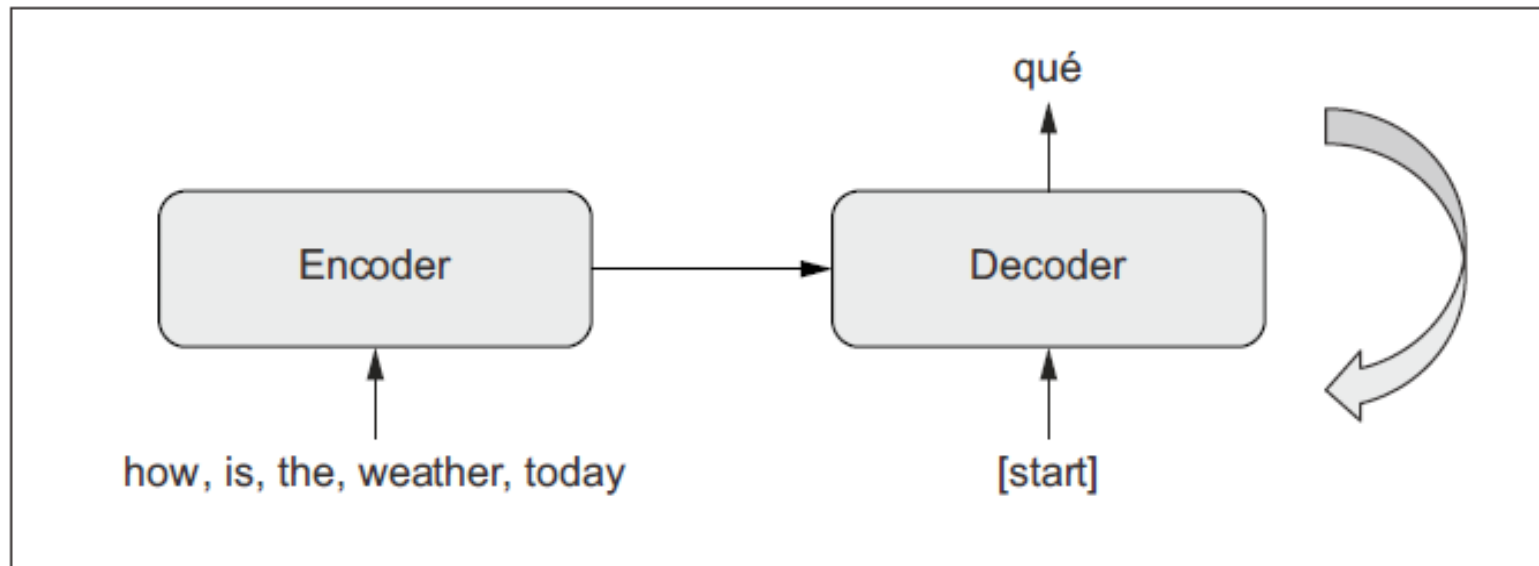
**Inference
phase**



Encoder + decoder

- During inference, we don't have access to the target sentence at all, since it's something we have to predict from scratch

**Inference
phase**



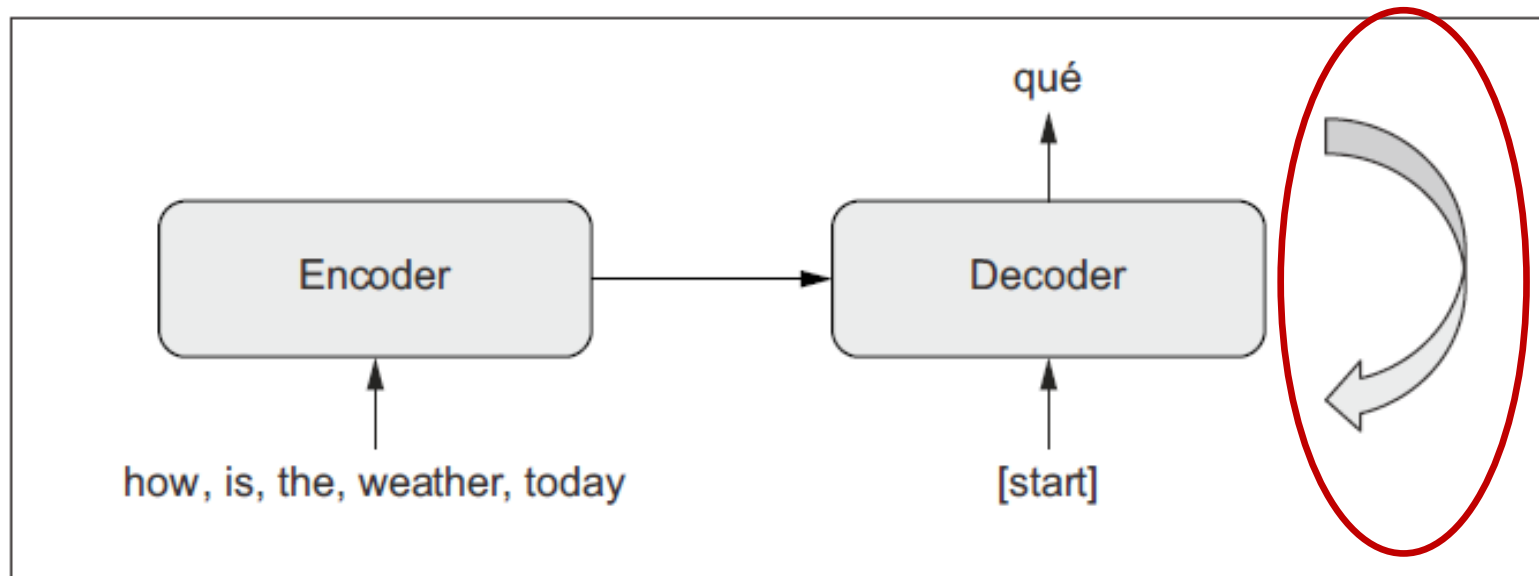
Encoder + decoder

- This target sequence will be generated one token at a time
- First, we have the encoder generate the embeddings for our source sequence
- The decoder will then receive this source sequence, and along with an initial seed token, e.g. “[start]”, use the info to predict the first actual token in the target sequence, e.g. “que”

Encoder + decoder

- The predicted token “que” is then fed back to the decoder, to generate the next token, and so on, until a stop token (e.g. “[end]”) is generated

**Inference
phase**



Encoder + decoder

- This is how the encoder + decoder works in tandem for machine translation
- Hopefully this allows us to see the different roles they play and how they interact with each other

Encoder – hands on

- Now, based on our discussion, we might notice that the encoder can actually be used to classify text, e.g. to predict if a review is positive or negative
- It's essentially a component that takes in an input sequence, and turns it into a more useful representation
- Let's go to our notebooks now to see if we can implement a Transformer encoder for predicting the sentiment of movie reviews

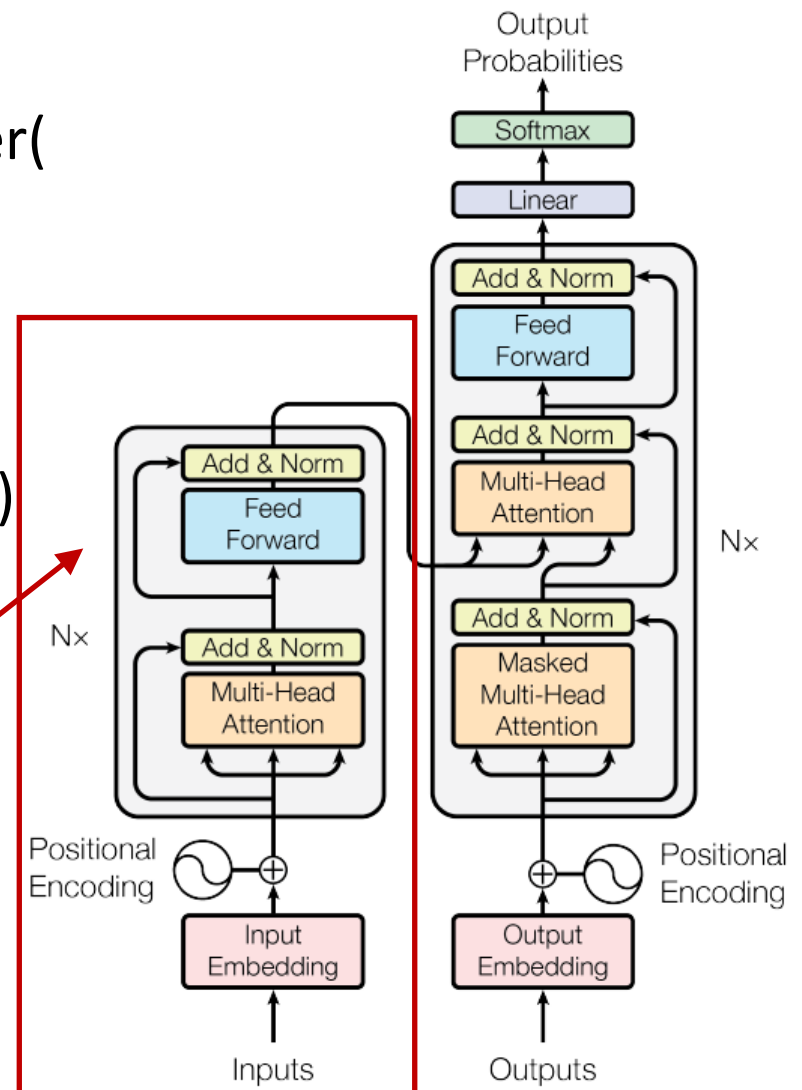
Encoder – hands on

- Create a new notebook on Kaggle
- Import the “Class3-1” notebook, and rename accordingly, and let’s go through the notebook

Specifying TransformerEncoder()

```
encoder =
keras_nlp.layers.TransformerEncoder(
intermediate_dim = layer_dim,
num_heads = num_heads,
kernel_initializer =
initializers.glorot_uniform(seed = 0))
```

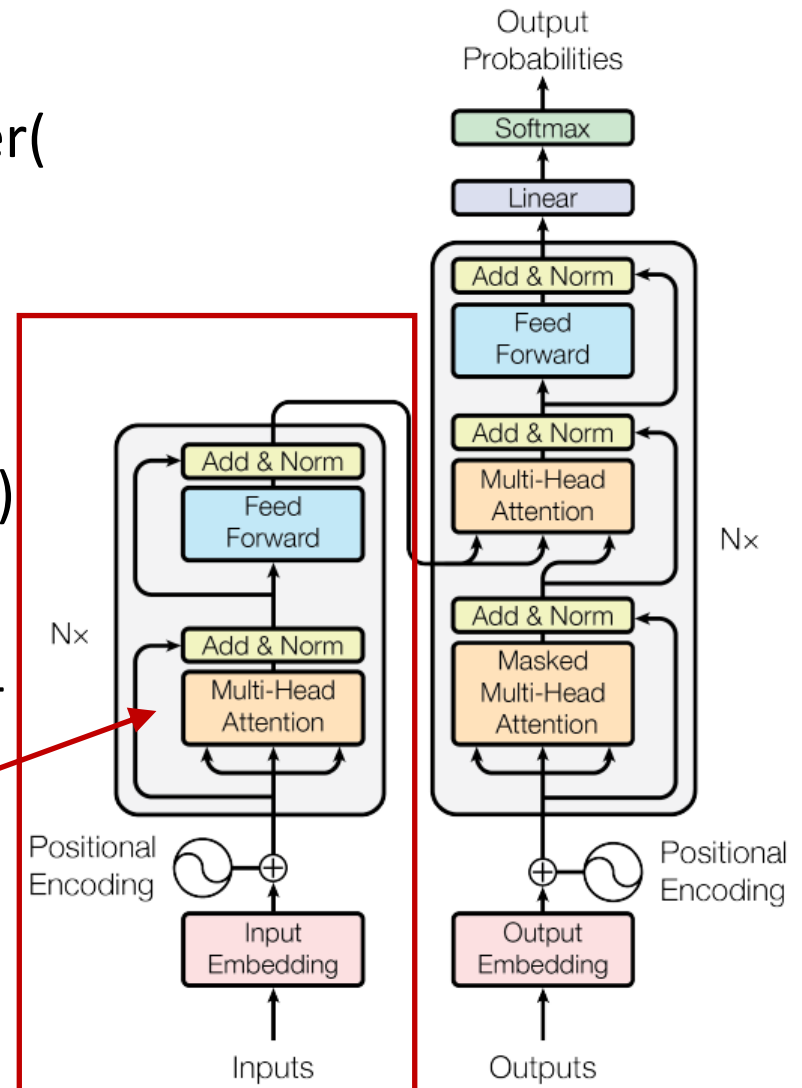
- This allows us to specify the number of nodes in the feed forward dense layer



Specifying TransformerEncoder()

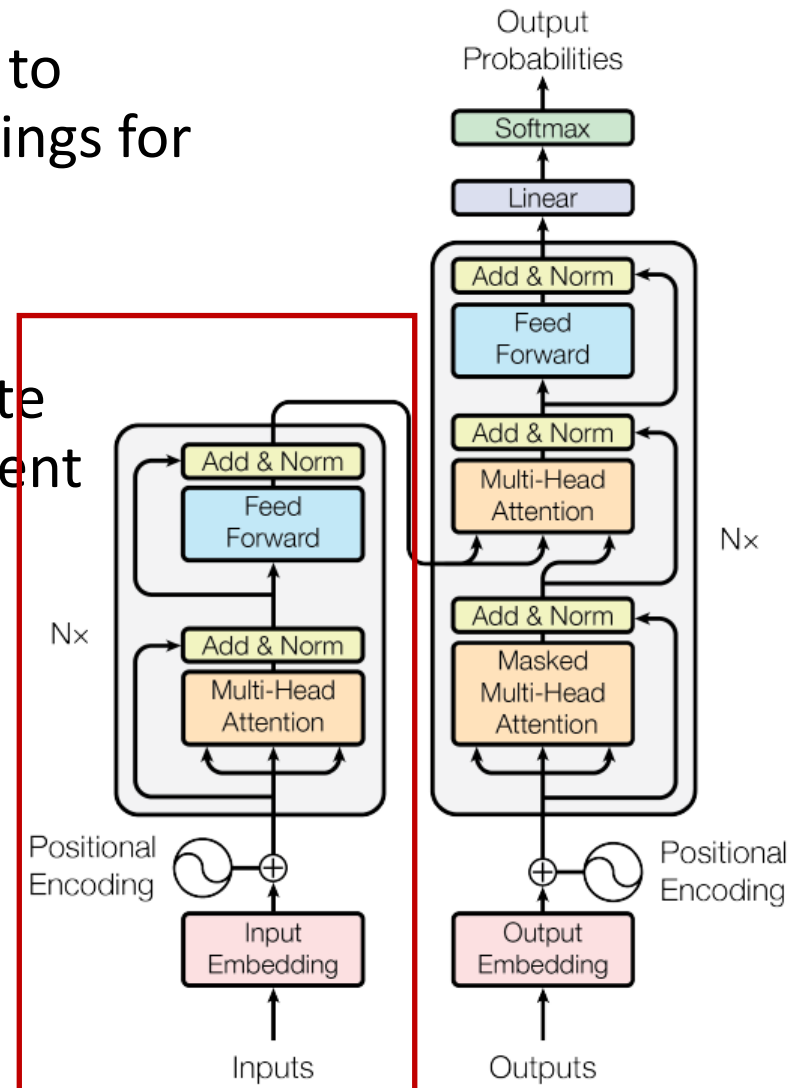
```
encoder =
keras_nlp.layers.TransformerEncoder(
intermediate_dim = layer_dim,
num_heads = num_heads,
kernel_initializer =
initializers.glorot_uniform(seed = 0))
```

- The number of heads in the multi-head attention layer



Specifying TransformerEncoder()

- The transformer encoder helps us to encode more meaningful embeddings for the movie reviews
- This will hopefully result in accurate predictions of the review's sentiment



Specifying TransformerEncoder()

- We will be passing batches of 32 reviews to the encoder for encoding, then predicting their sentiment
- This will be compared to the actual sentiment to get our loss, which is backpropagated and consequently have the weights adjusted to minimise the loss (repeated over the other batches in the same epoch, and for the other epochs)

Initializers

- `from keras import initializers`
- Initializers allow us to define the way to set initial random weights for our keras layers
- We can use the `kernel_initializer` argument to pass our initializer to our layer

Initializers

- encoder =
keras_nlp.layers.TransformerEncoder(intermediate_
dim = layer_dim, num_heads = num_heads,
kernel_initializer = initializers.glorot_uniform(seed
= 0))
- Here we specify how we wish the weights to be
initialized for our Transformer encoder layer

Masking

- Setting the argument: `mask_zero = True`, in the embedding layer
- This implements a padding mask used to mask out any padding tokens used in our input sequences, which does not contain meaningful information
- Recall that we used padding during our vectorization process, to pad to a length of 600 tokens per movie review

Masking

- So those reviews with words less than 600 will be post-padded with the 0 token, e.g.
- For instance, this was our first review

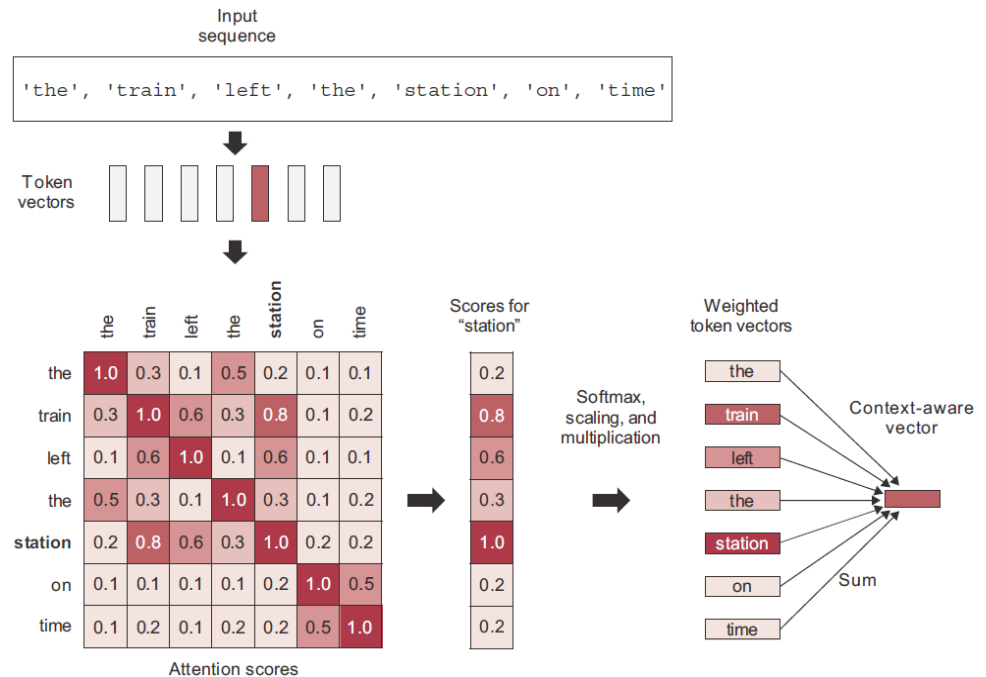
[illegible]

Masking

- The masking “hides” these meaningless tokens from the model, preventing the model from attending to them erroneously and affecting overall performance
- What about positional encoding?

Positional encoding

- We mentioned earlier that the Transformer is a sequence model, where word order is taken into consideration, unlike bag-of-words counterparts
- However, looking back at the attention layer...



Positional encoding

- We might notice that the tokens are being processed as a set – if we change the order of the tokens in the sentence, we would still get the same pairwise attention scores
- Recall in our class activity that we computed pairwise attention scores between Marvel characters, who are not in any order at all in a sentence

Positional encoding

- Consequently, the context-aware embeddings would also be the same, regardless of the word order
- Hence, self-attention really is looking at relationships between pairs of tokens, and it doesn't matter whether these tokens appear at the start, end, or middle of a sentence

Positional encoding

- So positional encoding is a way to manually inject word order into the sentences that the Transformer processes
- In positional encoding, we simply add the word's position in the sentence to each word embedding
- The most straightforward approach here would be to add a “position” axis to the word embedding vector, with 0 for the first word in the sequence, 1 for the second, etc

Positional encoding



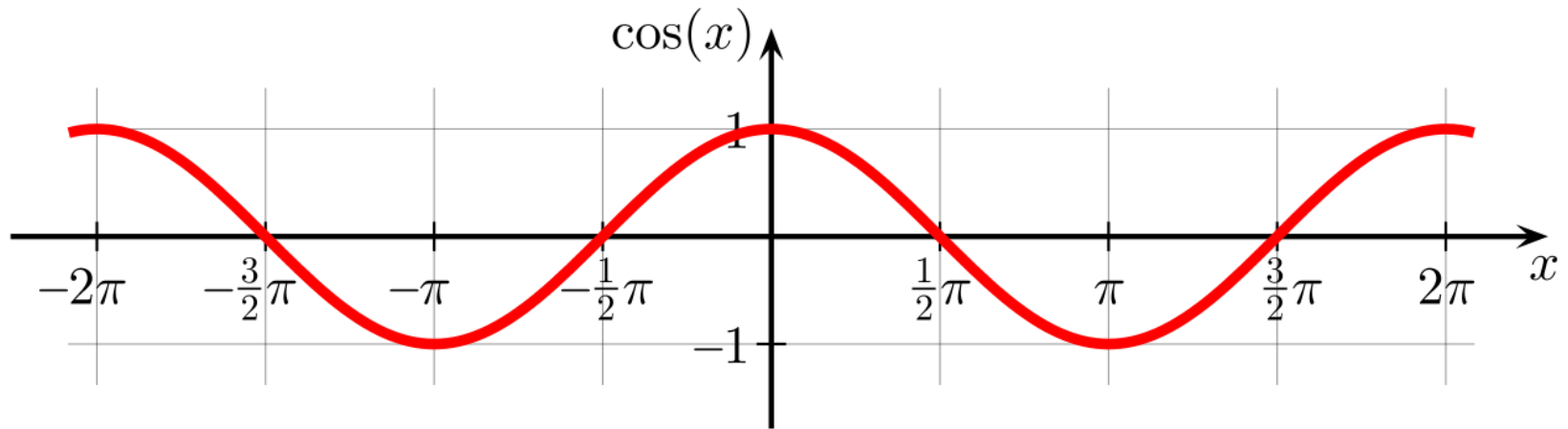
64 coef tall, in each
vector
representation of a
word; we add
another to
represent each
word's position. For
the first word, this
would be 0, for the
second word, this
would be 1, and so
on

600 words long

Positional encoding

- This is not very ideal approach as the positions can become very large integers, which might dwarf and disrupt the range of values in the embedding vector
 - E.g. if our longest review is 2000 words long, we will have a value of 1999 for that positional axis
- The approach that we adopt is the one in the “Attention is all you need” paper
- In this approach, we add to the word embeddings a vector containing values in the $[-1, 1]$ range, that varied cyclically according to the word position, based on sine and cosine functions

Positional encoding



In essence, what the positional encoder does is to get help of the cyclic nature of $\sin(x)$ and $\cos(x)$ functions to return information of the position of a word in a sentence

Positional encoding

- The specific sine and cosine functions used are the following:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Positional encoding

- Say we have a six-word sentence, “It is raining heavily outside today”, encoded as 4-dimensional embeddings, i.e. each word is a 4-dimensional vector
- I.e. $L = 6$ and $d_{\text{model}} = 4$

Positional encoding

- The following loop is used to generate our positional encoding matrix for all the words in our sequence, with n is just a scalar and is chosen to be 10,000

- For each $k = 0$ to L :
 - For each $i = 0$ to $\frac{d_{model}}{2}$:
 - $PE_{(k,2i)} = \sin\left(\frac{k}{\frac{2i}{n^{d_{model}}}}\right)$
 - $PE_{(k,2i+1)} = \cos\left(\frac{k}{\frac{2i}{n^{d_{model}}}}\right)$

Positional encoding

- So let's take a look at the first position, $k = 0$, i.e. "It"

- We iterate from $i = 0$ to $i = d_model/2 = 2$

- For $i = 0$: $PE_{0,0} = \sin(0 / 10000^{0/4}) = 0$

$$PE_{0,1} = \cos(0 / 10000^{0/4}) = 1$$

- For $i = 1$: $PE_{0,2} = \sin(0 / 10000^{2/4}) = 0$

$$PE_{0,3} = \cos(0 / 10000^{2/4}) = 1$$

- For each $k = 0$ to L
 - For each $i = 0$ to $\frac{d_{model}}{2}$:
 - $PE_{(k,2i)} = \sin\left(\frac{k}{n^{\frac{d_{model}}{2i}}}\right)$
 - $PE_{(k,2i+1)} = \cos\left(\frac{k}{n^{\frac{d_{model}}{2i}}}\right)$

So we see that the index i allows us to toggle between the sine function for odd positions in the positional encoding vector, and cosine functions for even positions

Therefore, the position encoding for this first word is $(0, 1, 0, 1)^T$

Your turn!

- Try calculating the position encoding vector for the second word, “is”

Positional encoding

- So we see that each word in our sentence gets a unique vector, based on its position relative to the other words
- Note that for any word position, given the same embedding dimension, the position vector will always be the same, e.g. the position vector for the first word in a sentence, for 4-dimension embeddings will always be $[0, 1, 0, 1]^T$
- As well it should be, since the vector is supposed to be reflective of word position alone; each position vector is then added to the original embedding vector for the word to get position-aware embeddings

Position encoding

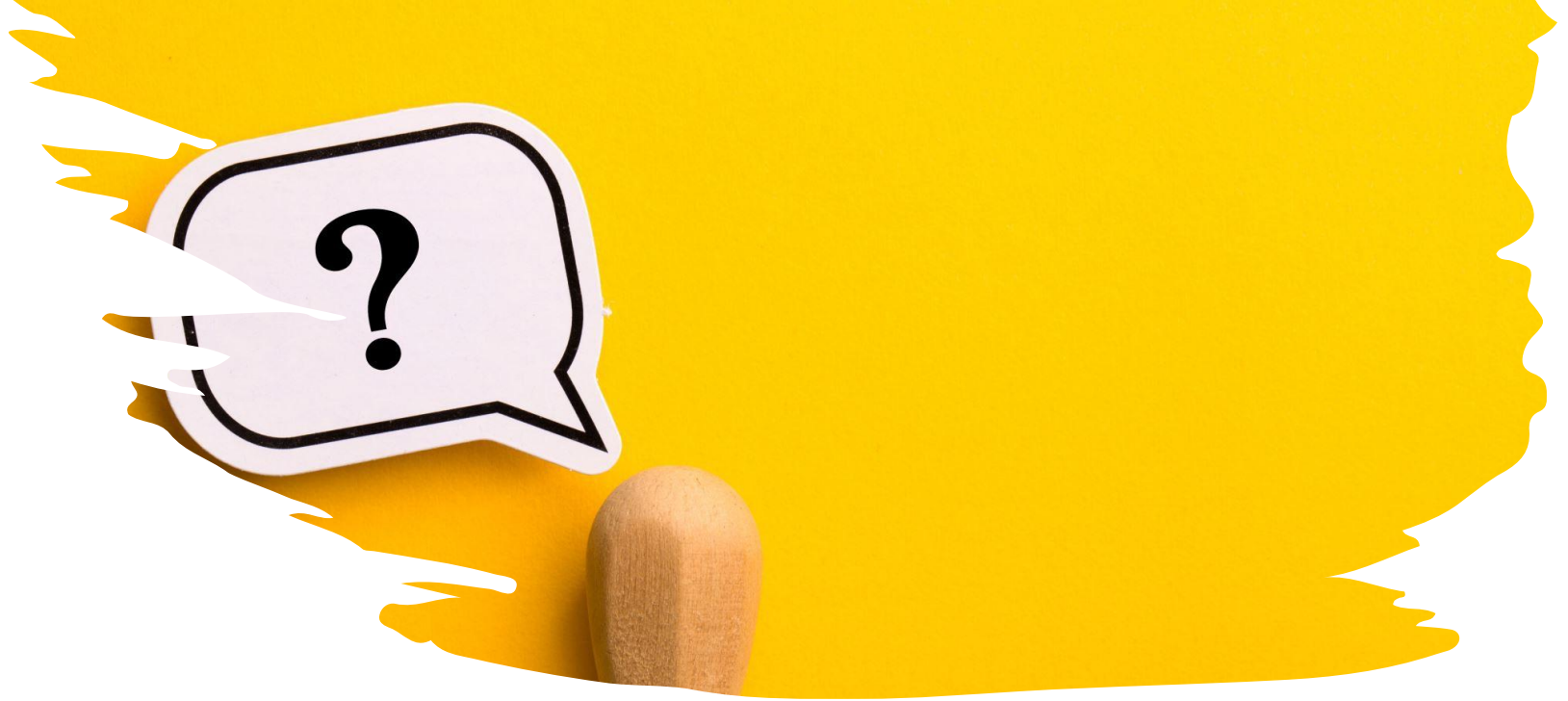
- Putting all the position vectors together into a position matrix, we get: (note the typo in the diagram)
- E.g. $PE_{0,3}$, which we calculate earlier, is the first row and fourth element, which is 1

$$PE_{(k,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

$$PE_{(k,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

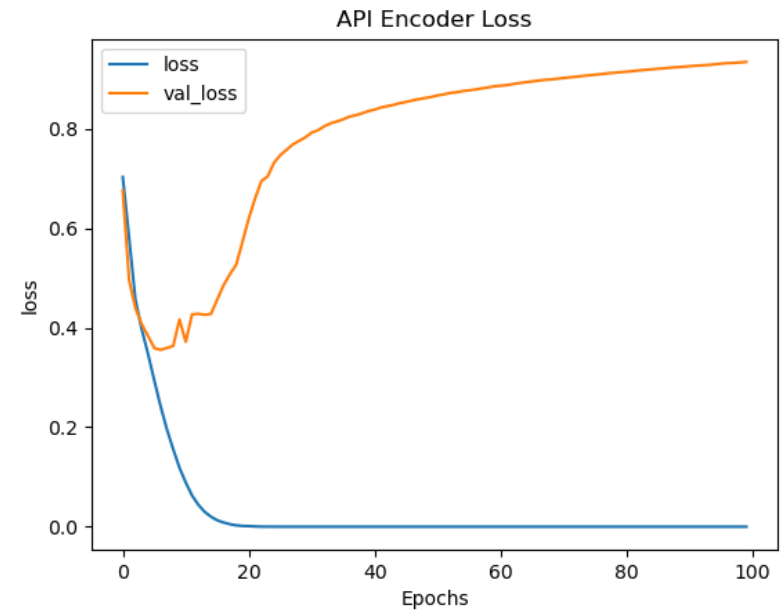
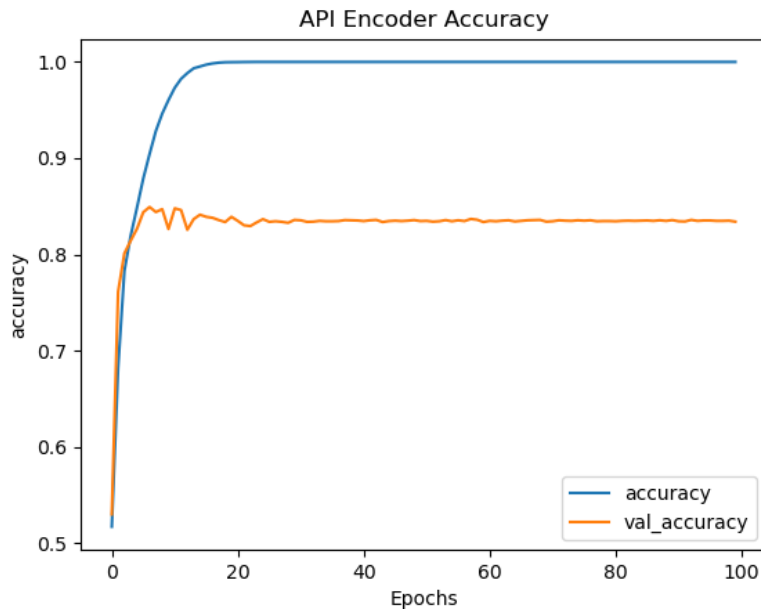
$$\begin{array}{c}
 \overbrace{d_{model} = 4} \\
 \left. \begin{array}{l}
 P(0) = \left[\sin\left(\frac{0}{10000^{\frac{0}{4}}}\right), \cos\left(\frac{0}{10000^{\frac{0}{4}}}\right), \sin\left(\frac{0}{10000^{\frac{1}{4}}}\right), \cos\left(\frac{0}{10000^{\frac{1}{4}}}\right) \right] \\
 P(1) = \left[\sin\left(\frac{1}{10000^{\frac{0}{4}}}\right), \cos\left(\frac{1}{10000^{\frac{0}{4}}}\right), \sin\left(\frac{1}{10000^{\frac{1}{4}}}\right), \cos\left(\frac{1}{10000^{\frac{1}{4}}}\right) \right] \\
 P(2) = \left[\sin\left(\frac{2}{10000^{\frac{0}{4}}}\right), \cos\left(\frac{2}{10000^{\frac{0}{4}}}\right), \sin\left(\frac{2}{10000^{\frac{1}{4}}}\right), \cos\left(\frac{2}{10000^{\frac{1}{4}}}\right) \right] \\
 P(3) = \left[\sin\left(\frac{3}{10000^{\frac{0}{4}}}\right), \cos\left(\frac{3}{10000^{\frac{0}{4}}}\right), \sin\left(\frac{3}{10000^{\frac{1}{4}}}\right), \cos\left(\frac{3}{10000^{\frac{1}{4}}}\right) \right] \\
 P(4) = \left[\sin\left(\frac{4}{10000^{\frac{0}{4}}}\right), \cos\left(\frac{4}{10000^{\frac{0}{4}}}\right), \sin\left(\frac{4}{10000^{\frac{1}{4}}}\right), \cos\left(\frac{4}{10000^{\frac{1}{4}}}\right) \right] \\
 P(5) = \left[\sin\left(\frac{5}{10000^{\frac{0}{4}}}\right), \cos\left(\frac{5}{10000^{\frac{0}{4}}}\right), \sin\left(\frac{5}{10000^{\frac{1}{4}}}\right), \cos\left(\frac{5}{10000^{\frac{1}{4}}}\right) \right]
 \end{array} \right\} k
 \end{array}$$

$i=0 \quad i=0 \quad i=1 \quad i=1$



Let's look at the results; what do we think?

These are the curves



Go back to our notebook; let's test our model on the new review we had in our last class

Bag-of-words vs sequence approaches

- It's not always that sequence models will outperform bag of words approaches
- A simple heuristic that we can refer to (Chollet), when selecting a model for text classification:

- $$\frac{\text{Number of samples}}{\text{Mean sample length}} \begin{cases} > 1500 \rightarrow \text{Sequence model} \\ < 1500 \rightarrow \text{Bag of words} \end{cases}$$

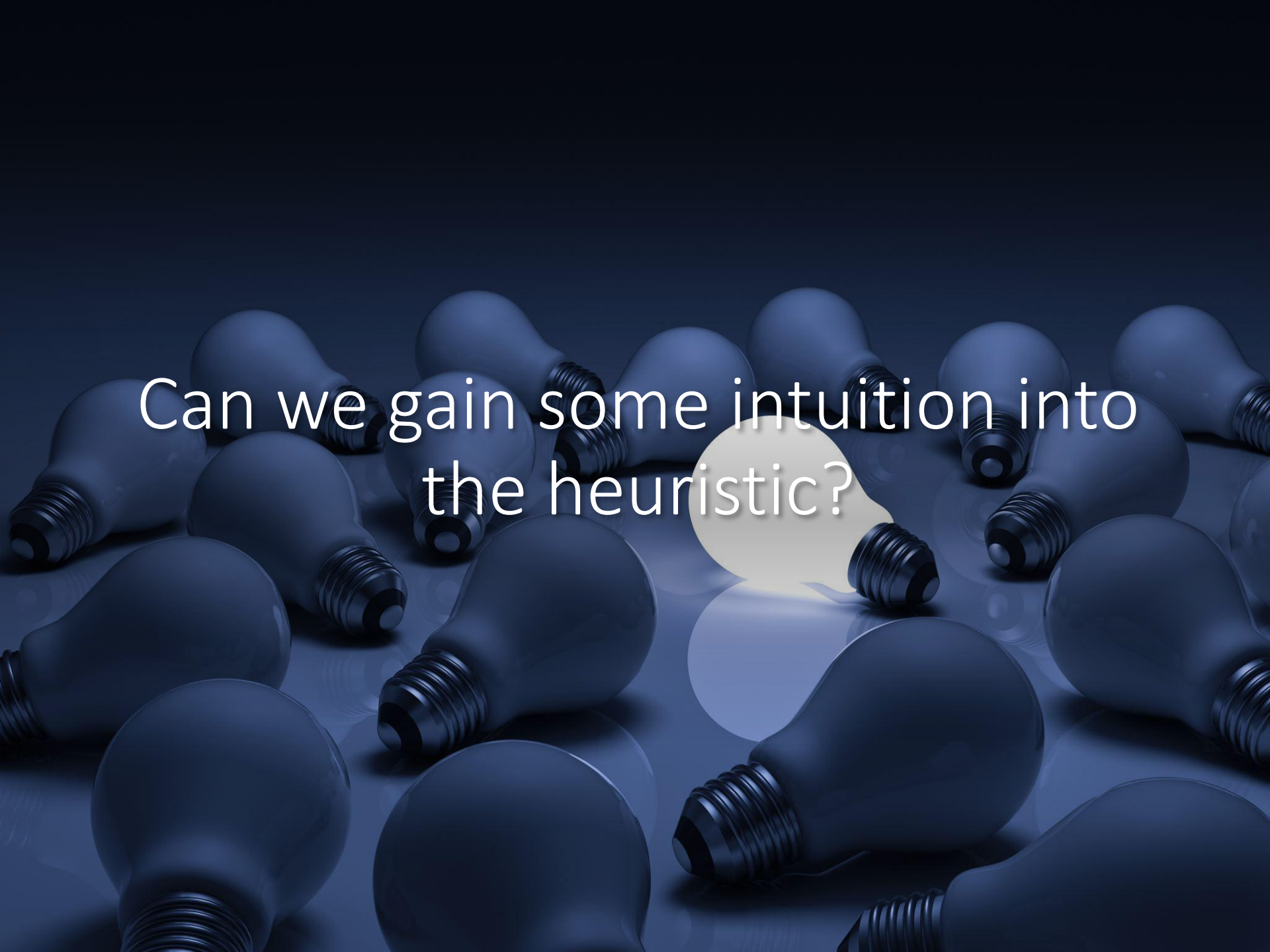
Based on this heuristic, what should we use?

Classifying (e.g. predicting the sentiment) of 1000-word long text, with a sample size of 100,000?

Classifying tweets which are 50 words long and we have 50,000 in our sample?

Our sample size for our tweets increases to 500,000?

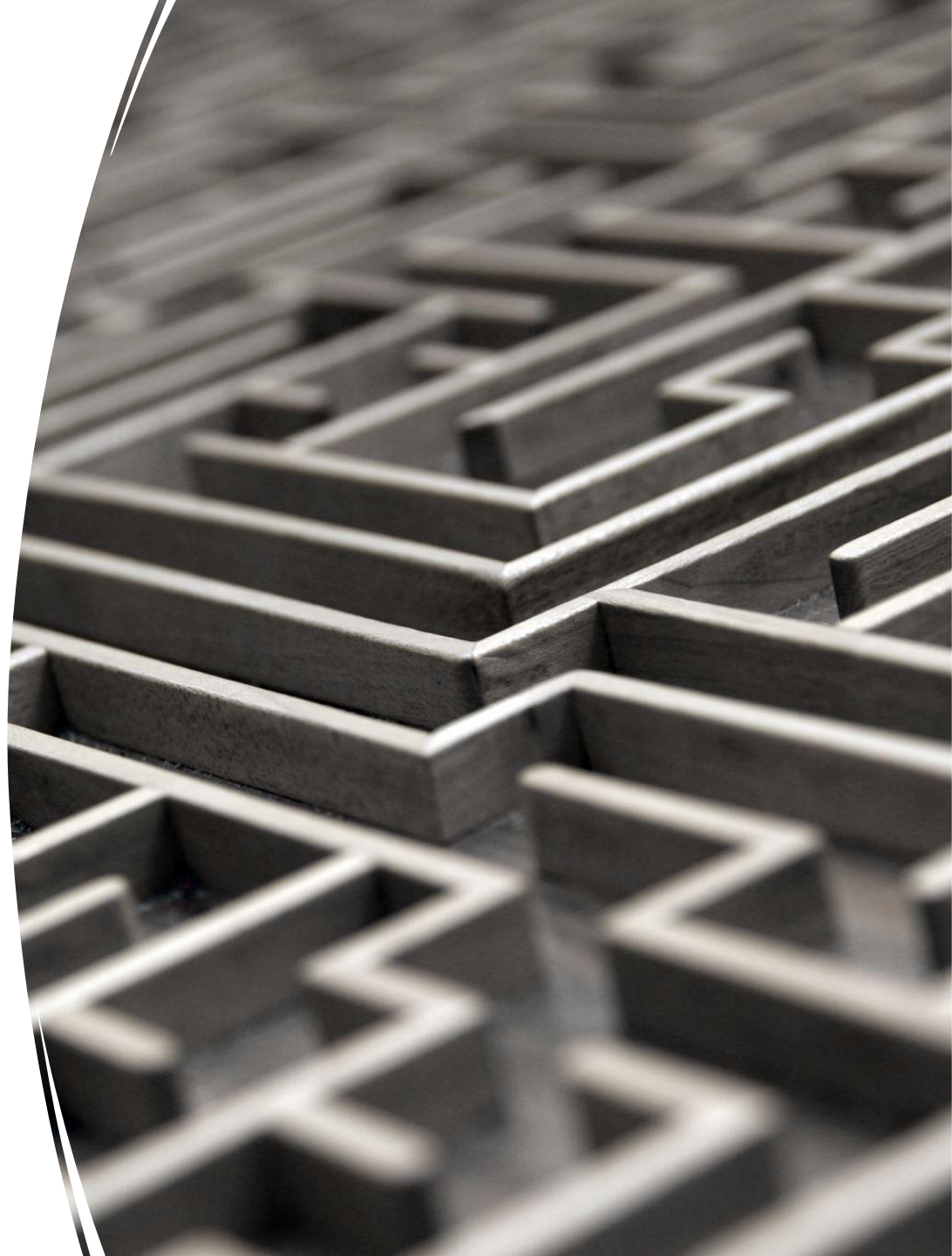
How about our movie reviews dataset?

A collection of light bulbs, with one glowing yellow in the center, surrounded by many unlit bulbs.

Can we gain some intuition into
the heuristic?

Next class

- Going beyond just predicting sentiment : looking at how our NLP models can be applied for text generation



End Day 3